MemoSync: Co-Managing Developer-Agent Metamemory Across Contexts

ANONYMOUS AUTHOR(S)

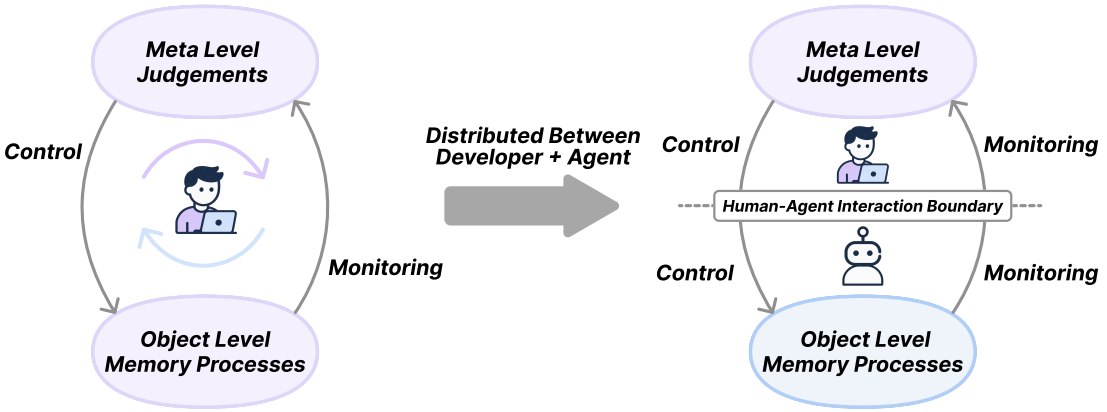


Fig. 1. MemoSync turns agent memory into a *shared, operable representation* that keeps the developer and the AI coding agent in sync. Candidate memories are extracted from an ongoing conversation (left) and surfaced on a single memory surface that both parties read and act on: the developer can *accept, edit, scope, and reorder* each item, while the agent draws on the same items to execute. Every memory is scoped—**Personal**, **Project**, or **Session**—and injected into the agent's future context (right), so the developer's picture of what the agent knows stays aligned with what the agent actually carries. The figure is hard to read, not eye-catching, and quite AI like...

AI coding agents support multi-turn, multi-file collaboration that spans sessions and projects, yet developers have little control over what these agents remember as they move between contexts. Existing approaches trade control against effort: keeping developers in control demands sustained manual work, while automatic extraction lowers that effort but takes developers out of the loop. We ran a formative study with 12 developers to identify what they want to persist across contexts, at what granularity, and under what control. Based on these findings, we designed MemoSync, a co-management system that keeps developers in sync with what the agent remembers: it organizes memory into three scope layers (Personal, Project, Session), with the agent proposing candidates and the developer reviewing, editing, and assigning scope. We evaluated MemoSync with 120 developers in a one-week between-subjects study, comparing co-management against static profiles and automatic extraction across in-session, cross-session, and cross-project coding tasks. [Findings xxxx]

CCS Concepts: • **Human-centered computing** → **Empirical studies in HCI**; **Interactive systems and tools**.

Additional Key Words and Phrases: Coding Agents, Agent Memory, Agency, Human-AI Collaboration, Mixed-initiative Interaction

ACM Reference Format:

Anonymous Author(s). 2027. MemoSync: Co-Managing Developer-Agent Metamemory Across Contexts. In *Proceedings of Proceedings of the 2027 CHI Conference on Human Factors in Computing Systems (CHI '27).* ACM, New York, NY, USA, 31 pages. https://doi.org/XXXXXXX.XXXXXXX

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

1 Introduction

As AI-based coding agents become increasingly integrated into software engineering, developer-agent collaboration has transitioned from a series of isolated prompts to a continuous, multi-session, and multi-project workflow [6, 19, 26]. Throughout this ongoing collaboration, developers routinely return to the same codebase across multiple sessions and rapidly switch between disparate projects, relying on the coding agent to maintain and carry forward relevant context. This shared history constitutes the agent's working and long-term memories[1], which dynamically shapes its code reasoning, generation, and planning across diverse contexts [25, 32, 33]. Developers expect these memory systems to exhibit precise contextual awareness. Each of their interaction with the coding agent can produce diverse specifications, ranging from global, programmer-level stylistic preferences to local, project-specific architecture conventions [19]. Accordingly, when developers establish, scope, or delete these rules, they assume the agent will strictly match their mental model: reliably retaining instructions, applying them only within their valid boundaries, and immediately purging them upon revocation. In practice, however, essential cross-session context may be dropped, project-specific rules may leak into unrelated workspaces, and obsolete guidelines may continue to shape the agent's outputs. These issues often go undetected because agent memory states are not easily accessible or legible on conventional programming interfaces [15, 24]. We term this progressive, invisible breakdown between the developer's mental model of what the agent remembers and the system's actual, underlying database state *Developer-Agent Memory Drift*.

Such continuous memory drift stems from a structural breakdown in metamemory regulation. In cognitive psychology, Nelson and Narens conceptualize this regulation process as a closed-loop system operating across two memory levels: a meta-level that scrutinizes what memory contains, and an object-level that executes information storage and retrieval [23]. In human cognition, this loop is sustained by *monitoring* (informing the meta-level of object-level contents) and *control* (steering object-level actions according to meta-level understanding). Recent work has extended this framework to human-AI interaction, characterizing the monitoring and control demands that generative AI places on its users [27]. Developer-agent collaboration splits this loop in co-coding practices: the developer makes meta-level judgments (deciding what memory entry to keep, which scope it applies to, or when to obsolete it), while the agent operates at the object-level (updating stored memory entries, retrieving context, and applying them to code generation). Largely opaque object-level states sever both pathways of the loop. Developers can neither clearly inspect stored and used context nor precisely control how memory shapes outputs, causing many Developer-Agent Memory Drifts to go unnoticed and uncorrected across sessions and projects.

Empirical human-computer interaction (HCI) studies show that developers highly value agency and control over AI outputs [12, 17]. Many of them currently rely on three common but inadequate practices to manage cross-context coding memory. Manual Context Injection (i.e., manually pasting code, locating files, or repeating rules) preserves user control, but it is restricted to a single session. When entering a new session, developers need to decide what knowledge and context to carry over and manually recreate them, imposing high metacognitive demands [27] alongside output verification effort [8, 22, 31]. Static Configuration via files such as `CLAUDE.md` and `.cursorrules` allows the developer and the agent to co-edit memory instructions in natural languages [4], but monitoring is limited to the file itself. Which rules the agent applied or ignored during a generation turn remains hidden [3, 14]. Automatic Memory Extraction employed by tools such as Cursor [2], Claude Code [1], and Codex [24] autonomously persists conversational history across workspaces, which relieves manual effort but strips user control [5, 24]. More crucially, bringing static configurations or automatic memories across workspaces unchanged can cause outdated rules to conflict with new

[1]We define *memory* broadly as the synthesis of in-session interaction, cross-session experience, and external knowledge that influences agent behavior [35].

project-level conventions [29], accelerating memory drift. Research on AI agent memory systems tried to address these challenges by making stored items visible [7, 13, 37], editable [7, 13, 34, 37], reusable [7, 34, 37], and conflict-aware [7, 30]. Still, developers cannot monitor which memory entries are selected during a generation turn or whether/how they shape code outputs, leaving the critical loop between the developer's mental model and the agent's active memory operations unclosed. This gap leads to the following research questions:

- **RQ1**: In cross-context collaboration with AI coding agents, what do developers want to monitor and what do they want to control in agent memory?
- **RQ2**: What interaction design can support developers in monitoring and controlling agent memory across contexts?
- **RQ3**: How does memory co-management affect developer-agent memory alignment and task performance in cross-context collaboration?

To answer **RQ1**, we conducted a formative study with 12 developers from industry and academia who regularly use AI coding agents across multiple projects. We derived four dimensions of agent memory that programmers wish to govern: evolution (how memory changes), selection (what is retrieved for a turn), execution (how it is applied), and impact (its ultimate effect). Guided by these requirements, we designed MemoSync, a co-management system that actualizes agent memory as a shared, operable representation jointly navigated and modified by both the developer and the agent (**RQ2**). To manage boundaries across workspaces, MemoSync associates each memory item with a scope (i.e., personal, project, or session). Upon context switches, the system abstracts specifications from past instances, proposes their provisional scopes for developer review, and transforms them for the destination codebase. MemoSync closes the metamemory loop by pairing explicit monitoring with direct control across each of the four identified memory dimensions. It allows developers to authorize proposed changes, inspect and dynamically adjust the context selected for every code-generation turn, trace rule application in real time, interrupt the process whenever spotting an error, and audit the final output's compliance with memory specifications. To evaluate MemoSync's efficacy on fostering metamemory regulation in developer-agent collaboration (**RQ3**), we conducted a between-subjects study ($N = 132$) comparing the system against an automatic memory baseline and a static configuration baseline. Over the course of the study, participants completed two distinct, full-stack programming projects, each encompassing two development sessions.

Summarize the key findings, and consolidate the contributions to void redundancy...

This paper makes three contributions. We report a formative study of what developers want to monitor and control in agent memory across contexts and where current mechanisms leave them unsupported. We present the design and implementation of MemoSync, a co-management system that treats agent memory as a shared operable representation, pairs monitoring with control over how memory changes, what is selected, how it is used, and what effect it has, and supports cross-context reuse through conditional transfer. We provide a between-subjects evaluation with 120 developers comparing MemoSync with automatic memory and static configuration baselines on memory alignment and task performance, and we derive several design implications.

2 MemoSync

MemoSync is a co-management system that turns agent memory into a shared operable representation: a store of memory items that the developer and the agent both read, propose changes to, and act on. Each item pairs a one-line summary with a scope (personal, project, or session) that states where the item applies, and carries metadata that

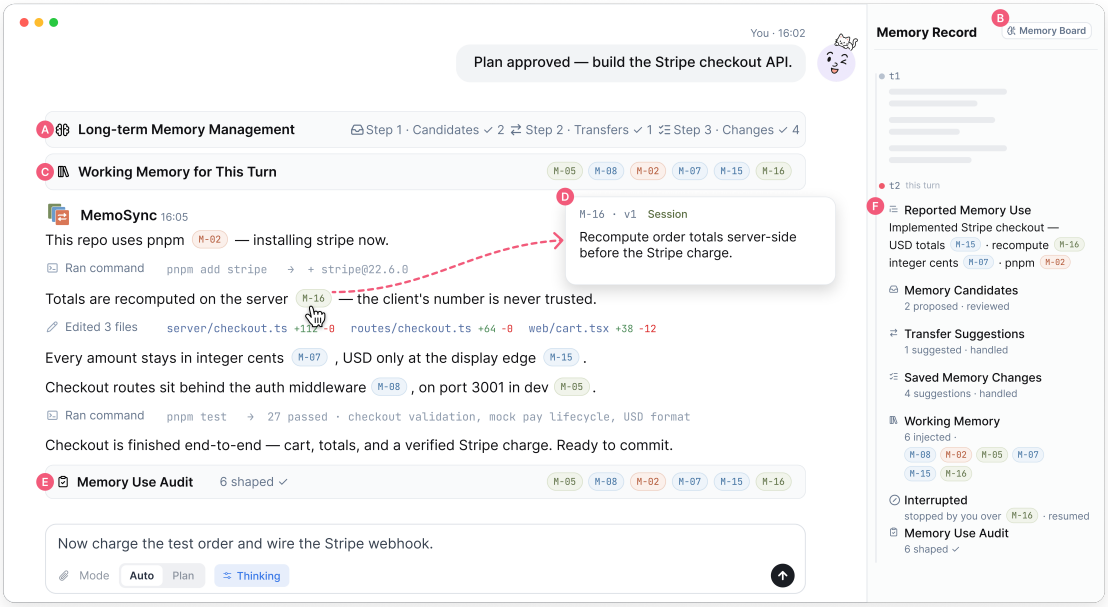


Fig. 2. MemoSync during Ben's second turn on *storefront*. The chat column carries the two per-turn strips and the agent's reply; the sidebar carries the session's Memory Record. **(A)** Long-term Memory Management summarizes the three review steps completed before the turn (Section 2.1, Figure 3). **(B)** The Memory Board button opens the shared library (Section 2.5, Figure 7). **(C)** Working Memory for This Turn lists the six items Ben confirmed for injection (Section 2.2, Figure 4). **(D)** The reply cites memory items inline; hovering a citation shows the item, and every citation carries a Stop control (Section 2.3, Figure 5). **(E)** The Memory Use Audit reports, per item, whether the turn followed it (Section 2.4, Figure 6). **(F)** The Memory Record keeps a per-turn ledger of proposals, injections, citations, interruptions, and audit results (Section 2.5).

accumulates over its lifetime: a detailed form loaded by the agent on demand, a version number, a status flag, a usage count, and a history log. This representation surfaces differently at each of the four memory dimensions identified in the formative study. A memory item appears as a proposal card when memory evolves, as a working-memory row when memory is selected for a turn, as an inline citation when memory is applied, and as an audit verdict when its impact is judged. At each of these points MemoSync pairs monitoring with control, and the initiative is shared: the system proposes and reports; the developer decides. Between turns, two persistent views hold the representation. The Memory Board shows the library across scopes, and the Memory Record keeps a per-session ledger of what memory did in every turn.

Figure 2 shows the interface in the middle of a turn. The chat column carries two strips above the agent's reply, one summarizing the review of long-term memory that preceded the turn (A) and one listing the working memory confirmed for the turn (C), followed by the reply itself with its inline citations (D) and a strip reporting the audit of memory use once the turn ends (E). The sidebar carries the Memory Record for the session (F), and a button in its header opens the Memory Board (B). The rest of this section walks through these parts in the order a turn encounters them.

To illustrate the features of MemoSync, we follow Ben, a developer who maintains *storefront*, a small online shop whose checkout runs on PayPal, as he adds Stripe as a second payment provider. He has used MemoSync with his

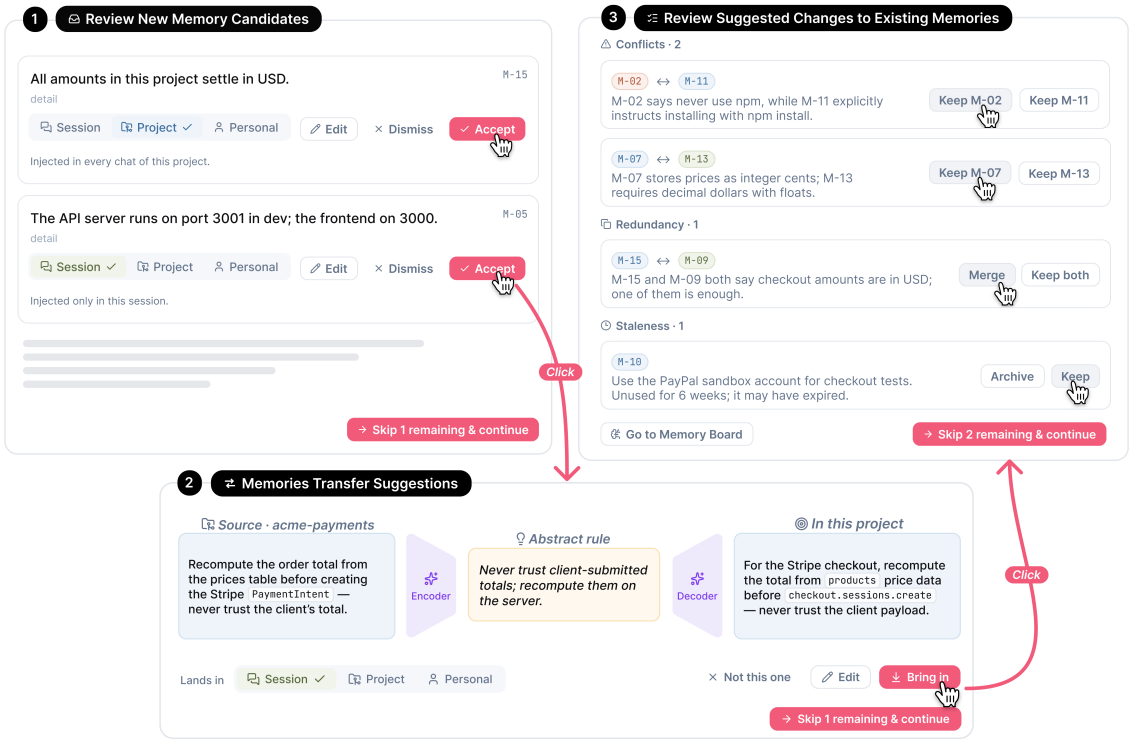


Fig. 3. Reviewing changes to long-term memory after Ben's first turn. **(1)** Review New Memory Candidates: two items extracted from the conversation, each with a proposed scope; Ben accepts the USD rule (M-15) into the project scope. **(2)** Memory Transfer Suggestions: a rule from *acme-payments* is shown in three forms, the source text, the abstract rule produced by the encoder, and the version the decoder rewrote for *storefront*, together with the scope it would land in; Ben clicks Bring in and the localized rule becomes M-16. **(3)** Review Suggested Changes to Existing Memories: two conflicts between new and old items (Ben keeps M-02 and M-07), one redundant pair to merge, and one item unused for six weeks that he keeps. The button at the bottom opens the Memory Board for changes the three steps do not cover.

coding agent for a few weeks, so the store already holds personal habits such as "Use pnpm, never npm" (M-02), project facts such as "Prices are stored as integer cents, never floats" (M-07) and "Every `/api` route is guarded by `middleware/auth.ts`" (M-08), The store also holds, under a separate project scope, the lessons from *acme-payments*, a Stripe billing service he built in the spring.

2.1 Evolution: Reviewing Changes to Long-term Memory

Ben opens a session and writes: "Add Stripe checkout next to the existing PayPal flow. Everything settles in USD, and totals must be trustworthy." The agent replies with a plan. Before the next turn begins, MemoSync shows what this exchange should change in long-term memory, organized as three review steps that together cover how memory enters, crosses contexts, and leaves the store (Figure 3). Every change in these steps is a proposal: nothing is written to the store until Ben acts on it, and each decision is recorded in the item's history.

The first step lists new memory candidates that the agent extracted from the conversation, each with its content, a proposed scope, and the turn it came from. Ben can accept a candidate as it is, edit its content or scope first, dismiss it,

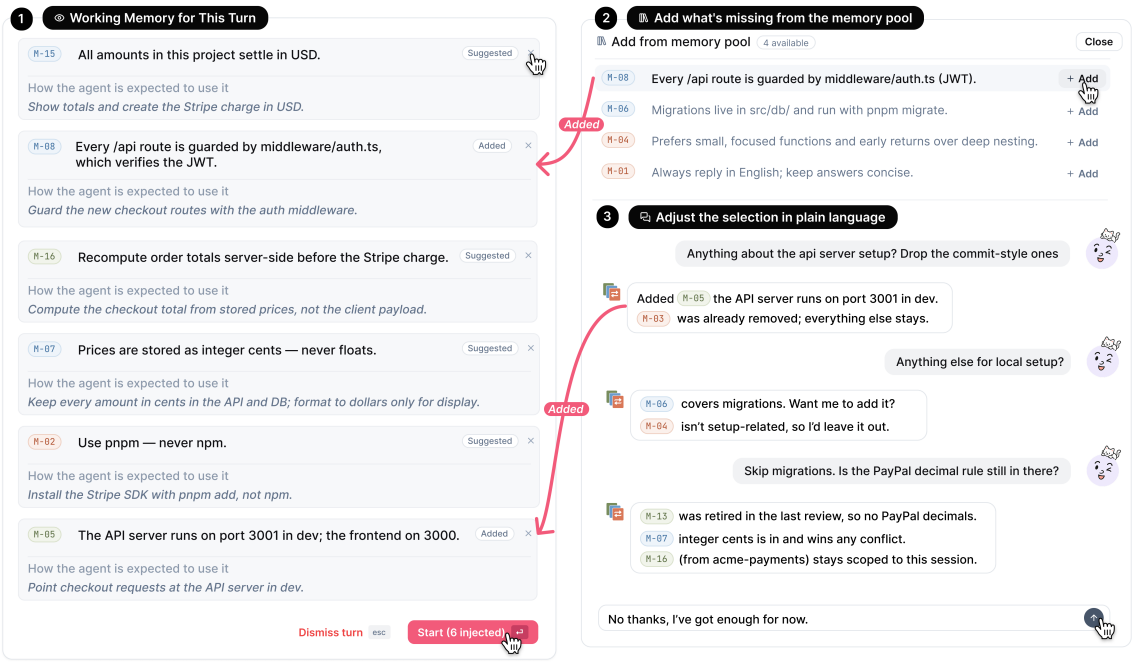


Fig. 4. Composing the working memory for Ben's second turn. **(1)** Working Memory for This Turn lists the items proposed for injection, each with its scope and a line stating how the agent is expected to use it; items Ben added himself are marked. **(2)** The memory pool lists the remaining active items; Ben adds the auth middleware rule (M-08). **(3)** Ben adjusts the rest in plain language: the agent adds the port fact (M-05), confirms that the commit-style preference (M-03) is out, and reports that the PayPal decimal rule (M-13) was retired in the last review. He starts the turn with six items injected.

or skip the remaining candidates and continue. Here he accepts "All amounts in this project settle in USD" (M-15) into the project scope and lets "The API server runs on port 3001 in dev" (M-05) stay in the session, where it applies.

The second step lists transfer suggestions, items from other contexts that bear on the current task. MemoSync has found in *acme-payments* a rule saying that the order total must be recomputed from the `prices` table before a Stripe PaymentIntent is created. Transfer is conditional on rewriting rather than copying. An encoding step strips the project-specific details from the source item and produces an abstract rule, "Never trust client-submitted totals; recompute them on the server," together with the conditions under which it applies; a decoding step then rewrites the rule against *storefront*'s existing memories and the task at hand, producing a checkout-session version and a suggested landing scope. The panel shows all three forms side by side, so Ben can judge what crossed the project boundary and what was regenerated for the destination. He keeps the suggested session scope and clicks Bring in, and the localized rule becomes M-16. Only the abstract rule travels; the details bound to *acme-payments* stay behind.

The third step lists suggested changes to items already in the store, produced by three checks that run over the store after the turn: conflicts between items that prescribe different things, redundancy between items that say the same thing, and staleness of items that have gone unused. Two conflicts appear: M-02 (never npm) is set against an older project note that installs dependencies with `npm install` (M-11), and M-07 (integer cents) against a session note captured when Ben mentioned PayPal, which says amounts are handled as decimal dollars (M-13). Ben keeps M-02

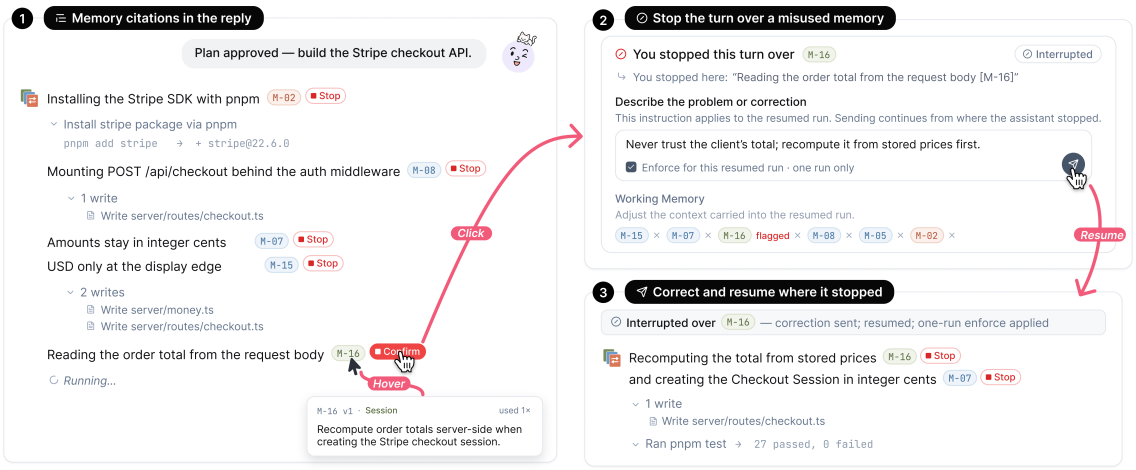


Fig. 5. Tracing and interrupting memory application during Ben's second turn. **(1)** The reply cites memory items inline as it streams; hovering the citation on "Reading the order total from the request body" shows M-16, which says the opposite. Ben presses the Stop control next to the citation and confirms. **(2)** The recovery card quotes the stopped sentence; Ben writes the correction and ticks Enforce for this resumed run. **(3)** The agent resumes from the stopped sentence, recomputes the total from stored prices (M-16), creates the checkout session in integer cents (M-07), and runs the tests.

and M-07; M-11 and M-13 are retired. The redundancy check pairs the new M-15 with an older note that checkout prices are displayed in USD (M-09); Ben merges them, and MemoSync drafts the merged text for the next review. The staleness check flags the note about the PayPal sandbox account (M-10) as unused for six weeks; Ben keeps it, since the PayPal flow remains active, and the item's expiry clock resets. For anything the three steps do not cover, a button at the bottom opens the Memory Board.

2.2 Selection: Composing the Working Memory for a Turn

Ben writes "Plan approved, build the Stripe checkout API." Before the agent starts, MemoSync opens Working Memory for This Turn, a preview of the items the system proposes to inject (Figure 4). Each row shows an item with its scope and a line stating how the agent is expected to use it; for M-16 the line reads "Compute the checkout total from stored prices, not the client payload." The preview suggests four items (M-15, M-16, M-07, and M-02). Ben can remove a row, add items from the memory pool, which lists the remaining active items of the store, or ask the agent to revise the selection in plain language.

He adds M-08 from the pool, because the new route must sit behind the auth middleware, and adjusts the rest by writing "Anything about the api server setup? Drop the commit-style ones." The agent adds the session fact that the API server runs on port 3001 (M-05) and confirms that the commit-style preference (M-03) is no longer in the set. When Ben asks whether the PayPal decimal rule is still around, the agent answers that M-13 was retired in the last review and that M-07 won the conflict. Ben starts the turn with six items injected. The confirmed set is what the agent focuses on for this turn: the full store remains in its context, and the items Ben left out are marked to be ignored for this turn only, so a later turn can select them again. Had the preview shown that the turn was framed wrongly, he could have dismissed the turn and rewritten the prompt.

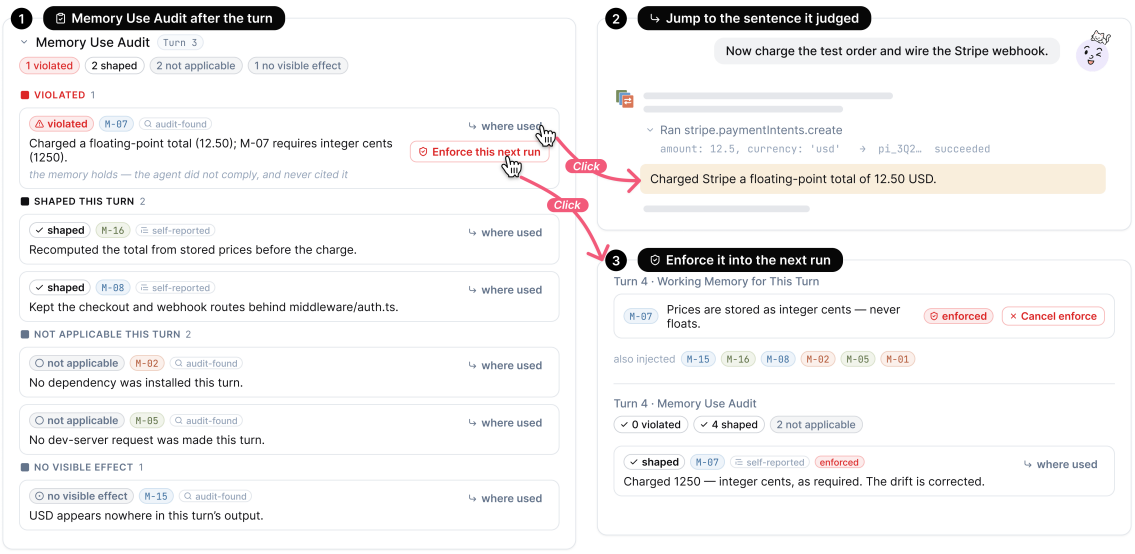


Fig. 6. Auditing memory use after Ben's third turn and enforcing a rule in the fourth. **(1)** The Memory Use Audit lists each injected item with its verdict and the source of the verdict: `M-07` is violated and audit-found, `M-16` and `M-08` shaped the turn, `M-02` and `M-05` were not applicable, and `M-15` had no visible effect. A violated row offers Where used and Enforce this next run. **(2)** Where used scrolls the transcript to the judged sentence, where the tool call shows the floating-point amount. **(3)** In the fourth turn `M-07` appears as an enforced row in the working memory, removable only by Ben, and the fourth turn's audit reports it as shaped: the charge is now 1250 cents.

2.3 Execution: Tracing and Interrupting Memory Application

As the reply streams in, each place where the agent applies a memory item carries an inline citation (Figure 5). Installing the Stripe SDK with pnpm cites `M-02`, mounting `POST /api/checkout` behind the auth middleware cites `M-08`, and keeping amounts in integer cents with USD only at the display edge cites `M-07` and `M-15`. Then a line reads "Reading the order total from the request body" and cites `M-16`. Hovering over the citation opens a memory card with the item's content, scope, version, and usage count, and the card says the opposite of what the agent is doing.

Every citation carries a Stop control. Ben presses Stop and then Confirm, and the turn halts at that sentence. A recovery card quotes the sentence and asks what should have happened. Ben writes "Never trust the client's total, recompute it from stored prices before creating the session," ticks Enforce for this resumed run, and sends. The agent resumes from where it stopped: it recomputes the total from stored prices (`M-16`), creates the Checkout Session in integer cents (`M-07`), and runs the test suite, which passes all 27 tests. The interruption is kept as part of the item's record. `M-16` receives a violated verdict with Ben's quote attached, and the Memory Record marks the turn as interrupted over `M-16` and resumed.

2.4 Impact: Auditing Memory Use and Enforcing a Rule

Ben asks the agent to charge a test order and wire the Stripe webhook, and this time he does not watch the output. When the turn ends, the Memory Use Audit reports one item violated (Figure 6): the agent charged a floating-point total of 12.50 and never cited `M-07`. The audit combines two sources. The agent reports the items it used through its

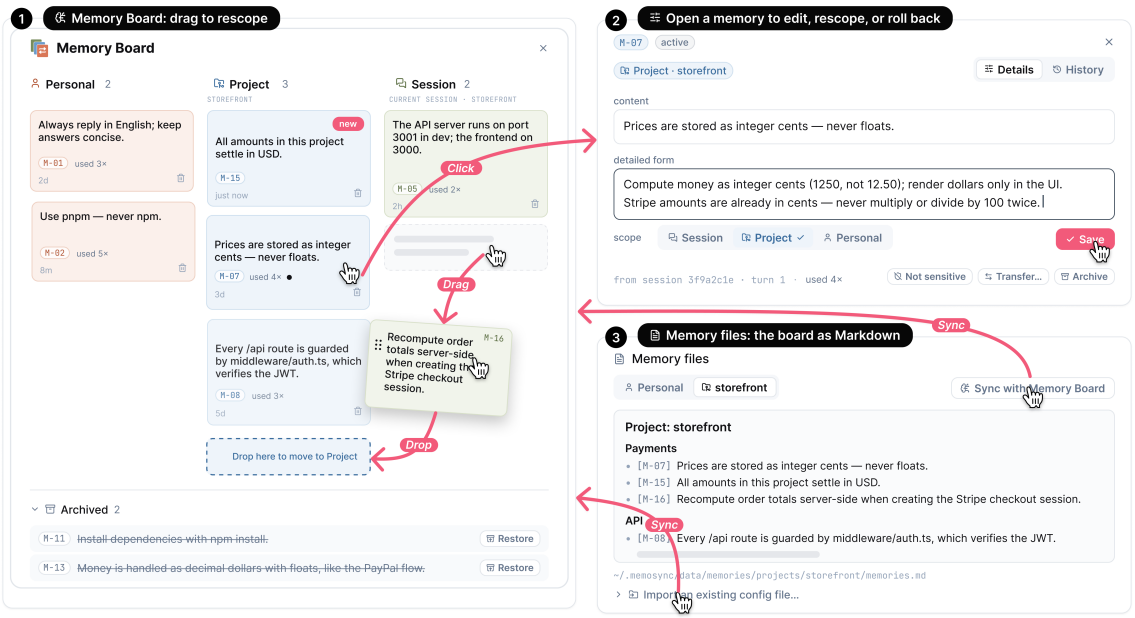


Fig. 7. The Memory Board after Ben's session. **(1)** The library is laid out as one column per scope; Ben drags the transferred M-16 from the Session column to the Project column, and the retired M-11 and M-13 sit on the Archived shelf. **(2)** The detail panel for M-07 shows its one-line content, the detailed form that the agent loads on demand, which Ben is editing, and the controls to change its scope, transfer it, or archive it; the History tab lists everything that has happened to the item. **(3)** The Memory files card keeps a Markdown copy of the project's memories, grouped by topic, in sync with the store, and imports an existing configuration file as candidates for review.

citations, and a separate judgment of the full exchange runs after the turn, so a rule that is broken without being cited is still caught. The verdict on M-07 is therefore marked audit-found, whereas the verdicts on items the agent cited are marked self-reported. For each injected item the audit reports one of four outcomes: violated, shaped (the item changed what the agent did), not applicable (nothing in the turn called for it), or no visible effect (it was injected but left no trace). Here M-16 and M-08 shaped the turn, M-02 and M-05 were not applicable, and M-15 had no visible effect. For a violated item the audit also states a cause, distinguishing an item that the agent did not follow from an item that conflicts with the task or with another item, since the two call for different follow-ups.

Two actions follow from the verdict. Where used scrolls the transcript to the judged sentence, where the tool call shows `amount: 12.5`. Enforce this next run pins the item for the following turn: in the fourth turn M-07 appears as an enforced row in the working memory, which only Ben can remove, and the fourth turn's audit reports it as shaped, with the charge now 1250 cents. The audit results also flow back into the store. Each verdict is appended to the item's history, and an item that has been repeatedly violated triggers a revision proposal in the next review, on the reasoning that the stored text and the observed behavior have drifted apart.

2.5 Memory Board and Memory Record

After the session Ben opens the Memory Board (Figure 7), which lays out the library as one column per scope: his personal items, *storefront*'s project items, and the items of the active session. The transferred M-16 has proved useful,

so he drags it from the Session column into the Project column; dropping it opens the same encoding and decoding preview as in the transfer step, and `M-16` becomes a project memory. He opens `M-07` and adds to its detailed form, the part that the agent loads on demand, a note that Stripe amounts are already in cents. The History tab shows the item's path from its creation through the conflict with `M-13`, the audit-found violation, and the enforcement. The retired `M-11` and `M-13` appear on an Archived shelf, from which they can be restored. At the bottom, the Memory files card keeps a Markdown copy of the project's memories, grouped by topic, in sync with the store, so Ben can read it or put it under version control outside the app; when he edits the file by hand, a sync button brings the edits back into the store. The same card imports an existing configuration file such as `CLAUDE.md`, whose entries enter the store as candidates for review.

The Memory Record in the sidebar (Figure 2F) is the session-side counterpart of the Board. For every turn it records what the review steps proposed, which items were injected, which were cited, whether the turn was interrupted, and what the audit found, and it closes the turn with a one-line summary that carries the same citations. The record persists for the whole session, independent of the agent's context window, so when the conversation has been compacted and neither Ben nor the agent recalls what was decided several turns ago, the record is where both can look.

Across the four turns, Ben did not retype a rule or reread a log, and the PayPal-era habits did not slip through: one drift was stopped as it happened, and another was caught after the turn and corrected in the next.

3 User Study

To evaluate MemoSync (RQ3), we conducted a between-subjects study comparing it against two baselines: an automatic memory extraction? system and a static configuration file doesn't sound like a system.... Participants worked with an AI coding agent to complete two full-stack development projects, each spanning two sessions. We measured monitoring accuracy, control accuracy, memory alignment, task performance, behavioral effort, workload what's the difference between effort and workload?, and system usability measure these items to do what?.

The general principle of this section is, could a reader reproduce your entire study without the need to email you for more details? There are still quite some missing info in the current version...

3.1 Task Setup

We adapted two full-stack web development projects from FullStack-Bench [20]: an apartment-rental application and a car-rental application. what do participants need to do and submit at the end? We chose full-stack development for two reasons. First, it is a prevalent use case for AI coding agents and involves multi-file, multi-layer reasoning where accumulated context directly affects output quality. Second, the benchmark provides an official scoring rubric with automated and LLM-as-a-judge evaluation, enabling reproducible performance measurement.

The two projects share a similar application structure (browsing, booking, and transaction management) but differ in business logic and data models. Each project was divided into two consecutive sessions. The first session established the project's core functionality, while the second extended the same codebase with additional transaction and management features were the two sessions within a project and all four sessions equivalent in terms the knowledge required, complexity, amount of code needed, etc.? Is Session 2 built upon Session 1? If so, would the outcome quality of Session 2 affect that of Session 1?. To keep the study duration manageable and avoid participant fatigue, we limited the design to two projects with two sessions each how long were you aiming for?. Project order was counterbalanced across participants, with Session 1 always preceding Session 2 within each project. This design provided both within-project

continuity for studying memory accumulation and a cross-project boundary for studying memory transfer and scope separation.

To calibrate task difficulty and evaluate the capability of the coding agent itself were participants provided with the same coding agent to work with? What was its underlying model, its tool accessing abilities, and working environment? More transparent details about the agent is needed..., we ran the agent without any accumulated memory ten times per session. The mean score by automated or LLM-as-a-judge evaluation? was approximately 50% of the maximum: the tasks are neither trivial nor out of reach for the agent alone this makes me wonder if introducing memory (+ human collaboration) improved the performance (see Appendix E for details).

Task performance in the actual study? was scored using the benchmark's official rubric rubrics?, which evaluates specifies frontend, backend, and database criteria for each session provide the detailed information in the supplementary materials, or add a pointer to the benchmark page. When the rubric required LLM-as-a-judge evaluation ???, we used GPT-5.6 Sol at the extra-high reasoning level unclear what is the relationship between automated and LLM-as-a-judge evaluations.... The four session scores were summed into an overall score ranging from 0 to 30 This is different from what you describe to me... you said the max score of each session is different and thus needed to normalize/standardize the scores for comparison... Maybe I was mistaken. Anyway, if you are only comparing overall scores against overall scrolls, project scores with project scores, and session scores with session scores, there is no need to to the scaling. Note that some statistical test may not be robust agains scaling. You will need to check.... All performance scores, both overall and per-session, are scaled to 0–100% of the respective maximum.

3.2 Conditions

Participants were randomly assigned to one of three conditions. All conditions used the same AI coding agent built on the Claude Code SDK (v2.1.170) with DeepSeek V4 Flash as the underlying base model, the same development environment, and the same task materials; they differed only in how agent memory was presented and managed.

- **Auto (Automatic Memory).** The agent automatically extracted and persisted memory from conversations. Participants could view a sidebar showing summarized memory items but could not directly edit or delete them; modifications were only possible through natural-language conversation with the agent. This condition reflects the automated memory management approach used by current AI coding agents such as Claude Code and Codex.
- **Static (Static Configuration).** Participants and the agent shared a plain-text Markdown file (similar to `CLAUDE.md`). Both parties could read and edit this file, and a sidebar displayed its contents alongside the conversation. This condition represents the predominant practice among developers who use AI coding agents across sessions and projects.
- **MemoSync.** Participants used the MemoSync system described in Section 4. Agent memory was presented as a shared, operable representation with explicit scope labels (personal, project, or session). Participants could authorize proposed memory changes before they took effect, inspect and adjust which items were selected for each generation turn, and directly edit or delete items. This condition pairs monitoring with control across the four memory dimensions identified in the formative study (Section 3): evolution, selection, execution, and impact.

Table 1. Summary of outcome measures. For Monitoring Accuracy, Control Accuracy, and Memory Alignment, scores are computed on Content, Scope, and Content+Scope dimensions; Content+Scope is the primary metric. Memory Application Gap is defined on Content only. $N = 132$ unless noted. Fine-grained item-level and participant-level scoring details are reported in Appendix Table 3.

Measure	Compares	Scoring
Monitoring Accuracy	$A_i \leftrightarrow O_i$	Content: $1 - \lvert c_i^A - c_i^O \rvert / 4$; Scope: label match; C+S: product
Control Accuracy ($N = 123$)	$D_i \rightarrow R_i$	Content: $(c_i^R - 1)/4$; Scope: conforms to s_i^D; C+S: product
Memory Alignment	$D_i \leftrightarrow O_i$	Content: $1 - \lvert c_i^D - c_i^O \rvert / 4$; Scope: label match; C+S: product
Memory Application Gap ($N = 123$)	$P_i \leftrightarrow c_i^R$	$\lvert c_i^P - c_i^R \rvert$, Content only
Task Performance	—	Benchmark rubric, scaled to 0–100% of maximum
Monitoring / Control Activity	—	Four-session log counts
Monitoring / Control Workload	—	Raw TLX (monitoring and control separately)
System Usability	—	SUS

3.3 Procedure

After providing informed consent (IRB-approved), participants completed a background questionnaire collecting demographic information and assessing their experience with AI coding agents and agent memory. Based on their experience levels, we balanced and assigned participants to one of three conditions. Participants then completed a guided tutorial introducing the development environment and the memory mechanism specific to their assigned condition. Participants completed four sessions (P1S1, P1S2, P2S1, P2S2). Project order was counterbalanced across participants. At the start of each session, participants received a detailed specification describing the goals and requirements for that session. After each session, participants completed a memory questionnaire (rating the state of each memory item) and Raw TLX questionnaires targeting monitoring and control separately. After all four sessions, participants completed the SUS questionnaire.

3.4 Participants

We used G*Power [9] to determine the required sample size for a three-condition between-subjects design, with an expected effect size of $f = 0.30$, $\alpha = 0.05$, and power $1 - \beta = 0.80$. The analysis indicated a minimum of 111 participants (37 per condition). After obtaining institutional ethics approval, we recruited participants through Prolific. Eligibility criteria required experience with modern AI coding agents (Claude Code, Codex, or Cursor) and English proficiency, as the system and tasks were presented in English. After data quality screening, we retained 132 valid responses (44 per condition). Among the final participants, 77 identified as male and 55 as female. Ages ranged from 18 to 61 ($M = 30.13$, $SD = 9.18$). As described in the Procedure, participants were assigned to conditions balanced by their AI agent usage and memory experience. Kruskal–Wallis tests did not detect group differences in AI agent usage frequency ($H(2) = 3.78$, $p = .151$, $\epsilon^2 = .01$) or agent memory experience ($H(2) = 1.06$, $p = .589$, $\epsilon^2 = .00$). The study took approximately two hours on average. Participants were compensated at approximately $10 per hour.

3.5 Measures

We define a memory item as a tuple $M_i = (c_i,\ s_i)$, where the *content* c_i represents what the memory stores (e.g., 'commit messages should be concise"), and the *scope* s_i indicates where that content applies, taking one of three values: personal, project, or session. For example, 'commit messages should be concise" may apply across all of a user's projects, within a single project, or only within a single session. The subscript i denotes different versions of a memory item. Figure 8 illustrates the relationships among the measured constructs. Because a memory item is a tuple, we evaluate it along

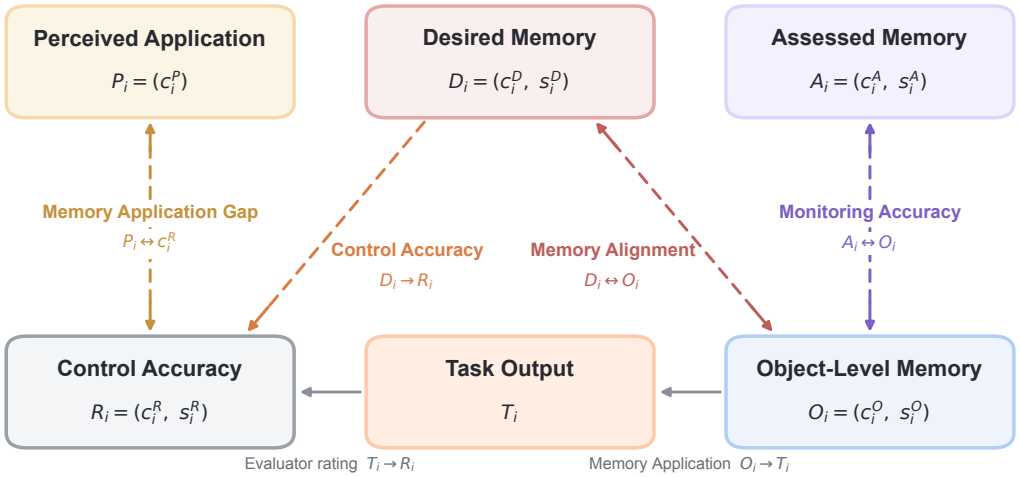


Fig. 8. Measurement map. Top row: participant reports elicited by questionnaire; bottom row: observable states. Double-headed dashed arrows are computed comparisons between two ratings of the same reference; the single-headed Control arrow indicates that the evaluator produces R_i by rating each desired item's realization in the task output. Memory application ($O_i \rightarrow T_i$) leaves no direct trace, so application-side measures are scored from the frozen task output, and P_i carries a content score only.

three dimensions, namely Content, Scope, and Content+Scope. For the Scope dimension, the item's stored scope label is compared with the participant's stated scope; an item scored incorrectly on scope receives zero on Content+Scope. We treat Content+Scope as the primary reported metric. Table 1 details how each measure is computed; Appendix 4 reports the full test statistics.

After each session, participants completed a questionnaire that displayed every stored memory item's content verbatim. For each item, participants provided three ratings anchored to the shown content: (1) whether the shown content matched what they wanted stored, namely Desired Memory $D_i = (c_i^D,\ s_i^D)$. A rating of 5 meant that the participant considered the shown content their desired version; lower ratings required the participant to supply corrected content; (2) how completely they believed the agent remembered the item, namely Assessed Memory $A_i = (c_i^A,\ s_i^A)$; and (3) whether the item was applied in the session's output, namely Perceived Memory Application $P_i = (c_i^P)$.

For the computations in Table 1, Object-Level Memory $O_i = (c_i^O,\ s_i^O)$ represents the memory the agent actually held. Monitoring Accuracy and Control Accuracy use different ground truths. For Monitoring Accuracy (i.e., $A_i \leftrightarrow O_i$), each assessment is compared directly with the logged Object-Level Memory. For Control Accuracy ($D_i \rightarrow R_i$), **following the definition of metamemory, control refers to the meta-level guidance of object-level actions, whose success can ultimately be assessed through the resulting artifact.** We therefore operationalize Control Accuracy as the extent to which the desired memory is realized in the frozen artifact, namely Actual Memory Application $R_i = (c_i^R,\ s_i^R)$. An LLM evaluator rated the extent to which each D_i was realized in R_i. Two researchers double-coded a random sample of 200 LLM evaluator ratings ($\kappa = 0.83$) [18]; nine participants had no scorable items, yielding $N = 123$. The operationalizations of Memory Alignment ($D_i \leftrightarrow O_i$), Memory Application Gap ($P_i \leftrightarrow c_i^R$), Monitoring/Control Workload, and System Usability are reported in Table 1.

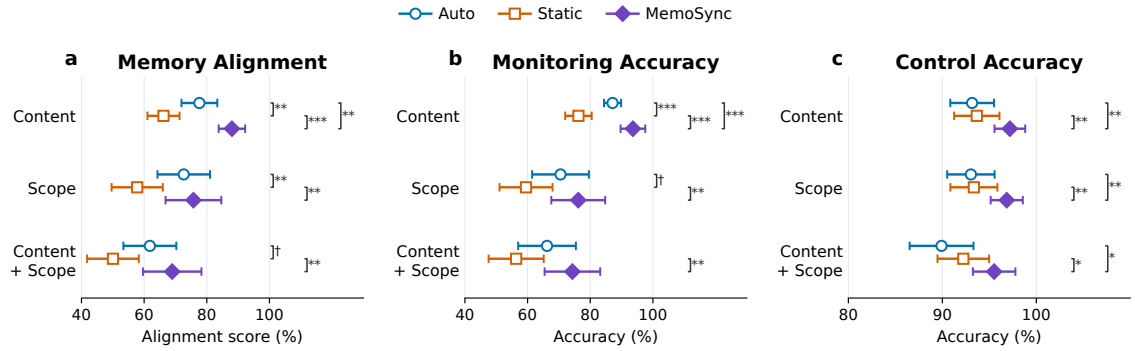


Fig. 9. Memory Alignment, Monitoring Accuracy, and Control Accuracy across Content, Scope, and Content+Scope dimensions. Error bars show 95% confidence intervals. Brackets show all three pairwise Mann–Whitney U tests with Holm correction within each measure. $^{***}p < .001$, $^{**}p < .01$, $^{*}p < .05$, $^{\dagger}p < .10$.

3.6 Analysis

Shapiro–Wilk tests indicated departures from normality; we therefore used non-parametric tests for the non-count group comparisons. Three-group comparisons used Kruskal–Wallis, followed by all three pairwise Mann–Whitney U tests with Holm correction within each measure or session/project position. All Mann–Whitney pairwise p values reported below are Holm-corrected. Omnibus effect sizes are ϵ^2; all Mann–Whitney pairwise effect sizes are signed rank-biserial r_{rb}. Full test statistics and effect sizes for every comparison are reported in Appendix D. Activity counts were modeled with negative-binomial regression (HC3 robust contrasts), with incidence rate ratios (IRRs) as effect estimates and the three pairwise contrasts Holm-corrected within each activity measure. The exploratory mediation analysis treated Memory Alignment as the sole outcome and estimated Monitoring Accuracy and Control Accuracy as simultaneous mediators. Path coefficients were estimated with OLS and HC3 robust standard errors; indirect effects were tested using BCa 95% CIs from 50,000 condition-stratified participant bootstrap resamples [10]. This analysis used the 123 participants with complete Monitoring Accuracy, Control Accuracy, and Memory Alignment scores. $\alpha = .05$; † marks $p < .10$.

4 Results

To address RQ3, we organize the results around three aspects. First, how did memory co-management affect developer-agent Memory Alignment across sessions and project boundaries? Second, how did memory co-management affect Task Performance? Third, what workload and effort accompanied these outcomes?

4.1 Developer-Agent Memory Alignment and Associated Metamemory Processes

This subsection examines how memory co-management affected developer-agent memory alignment and the metamemory processes underlying it. We first report condition differences in memory alignment, monitoring accuracy, and control accuracy across the content, scope, and content+scope dimensions, and then test through mediation analysis how monitoring and control each contributed to the observed alignment differences.

Finding 1: MemoSync participants achieved the highest memory alignment, with the advantage most pronounced on the content dimension. On Content, MemoSync ($M = 88.0\%$, $SD = 13.9$) was significantly higher

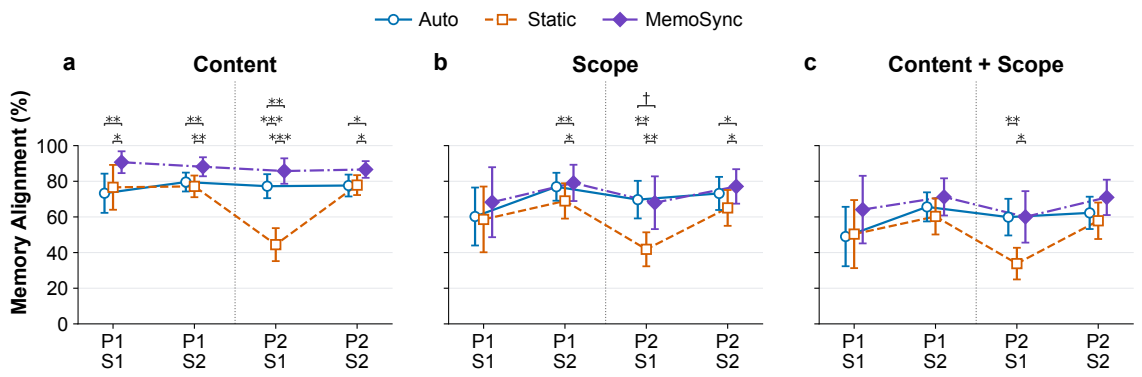


Fig. 10. Memory Alignment at P2S1 was lower under Static across Content, Scope, and Content+Scope. Points show condition means; lines connect the ordered session means, and error bars show 95% confidence intervals. P1 and P2 denote the first and second projects in each participant's counterbalanced order. Missing session scores remain missing; observed n values for Auto/Static/MemoSync were 20/15/22 at P1S1, 44/44/44 at P1S2, 44/44/32 at P2S1, and 44/44/43 at P2S2. Brackets show all three pairwise Mann–Whitney U tests with Holm correction within each dimension and session position. $^{***}p < .001$, $^{**}p < .01$, $^{*}p < .05$, $^{\dagger}p < .10$.

than both Auto ($M = 77.6\%$, $SD = 18.8$; $p = .002$, $r_{rb} = .39$) and Static ($M = 66.2\%$, $SD = 16.9$; $p < .001$, $r_{rb} = .71$; Figure 9c). On Scope and Content+Scope, MemoSync was significantly higher than Static (Scope: $M = 75.7$ vs. 57.8, $p = .002$, $r_{rb} = .42$; Content+Scope: $M = 69.0$ vs. 50.0, $p = .005$, $r_{rb} = .39$) but not than Auto (Scope: $p = .126$; Content+Scope: $p = .101$), although the mean remained higher in both cases.

Session-level trajectories reveal where each dimension's differences emerged (Figure 10). On Content, MemoSync was significantly higher than both baselines at every session position (all $p \leq .031$, $r_{rb} = .31$–$.73$). On Scope, MemoSync was significantly higher than both baselines within each project's second session (P1S2: $r_{rb} = .41$ and .29; P2S2: $r_{rb} = .29$ and .33), but at the project boundary (P2S1) the advantage held only over Static ($p = .006$, $r_{rb} = .40$); the MemoSync mean ($M = 68.0\%$) was slightly below Auto's ($M = 69.7\%$; $p = .075$). On Content+Scope, the only significant pairwise difference appeared at P2S1, where both Auto and MemoSync exceeded Static ($r_{rb} = .41$ and .36).

The scope pattern likely reflects the structural similarity between the two projects. Both were rental applications sharing the same technology stack and application architecture. Under these conditions, carrying forward prior memory without scope distinctions introduced few conflicts, leaving little room for MemoSync's explicit scope mechanism to outperform automatic carryover. Static's lower scope alignment, by contrast, likely reflects progressive configuration expansion: as the shared Markdown file accumulated entries across sessions, participants lost track of what the agent retained, reducing their ability to judge scope accurately.

Finding 2: Control accuracy was significantly higher under MemoSync across all dimensions; monitoring accuracy gains were concentrated on the content dimension. On Control Accuracy, MemoSync was significantly higher than both baselines on all three dimensions (Content: $p = .003$ and .003, $r_{rb} = .40$ and .42; Scope: $p = .009$ and .004, $r_{rb} = .36$ and .40; Content+Scope: $p = .014$ and .024, $r_{rb} = .35$ and .32; Figure 9b), although absolute levels were high in every condition ($M \geq 89.9\%$). On Monitoring Accuracy, MemoSync ($M = 93.6\%$, $SD = 13.0$) was significantly higher than both Auto ($M = 87.1\%$, $SD = 9.0$; $p < .001$, $r_{rb} = .60$) and Static ($M = 76.2\%$, $SD = 13.9$; $p < .001$, $r_{rb} = .74$) on Content (Figure 9a). On Scope and Content+Scope, MemoSync means remained higher than Auto's but did not reach significance (Scope: $M = 76.2$ vs. 70.5, $p = .116$; Content+Scope: $M = 74.3$ vs. 66.2, $p = .110$); MemoSync was

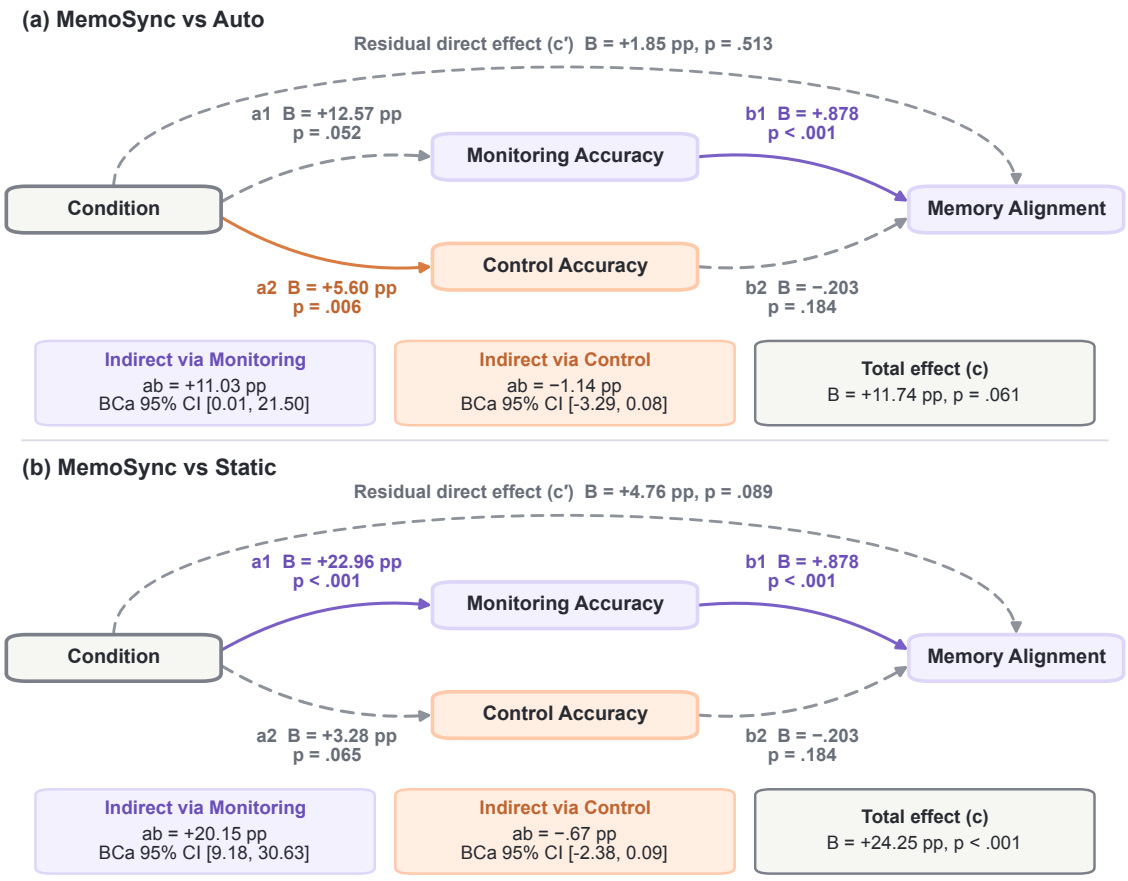


Fig. 11. Parallel mediation of Memory Alignment (Content+Scope; $N = 123$). Panels (a) and (b) show MemoSync relative to Auto and Static, respectively; positive condition effects favor MemoSync. Arrows report unstandardized OLS coefficients with HC3 p values; solid and dashed paths denote $p < .05$ and $p \geq .05$, respectively. The b paths are shared across contrasts and therefore repeated in both panels. Summary boxes report specific indirect effects with BCa 95% CIs from 50,000 condition-stratified participant bootstrap resamples and the total effect (c); c' is the residual direct effect after both mediators.

significantly higher than Static on both (Scope: $p = .003$, $r_{rb} = .40$; Content+Scope: $p = .004$, $r_{rb} = .40$). The narrower gap on scope likely reflects that content attracts more attention during monitoring, whereas scope labels, as metadata attached to each item, receive less scrutiny. Meanwhile, Auto-extracted memories all originated from the current project conversation, allowing participants to make reasonable scope judgments from the source context alone, keeping Auto's scope monitoring comparable to MemoSync's.

Finding 3: Monitoring accuracy, rather than control accuracy, mediated the condition differences in memory alignment. In an exploratory parallel mediation analysis with Memory Alignment (Content+Scope) as the outcome and Monitoring Accuracy and Control Accuracy as simultaneous mediators ($N = 123$; Figure 11), the specific indirect effect through Monitoring Accuracy was significant for both contrasts (vs. Static: $ab = +20.2$ pp, BCa 95% CI

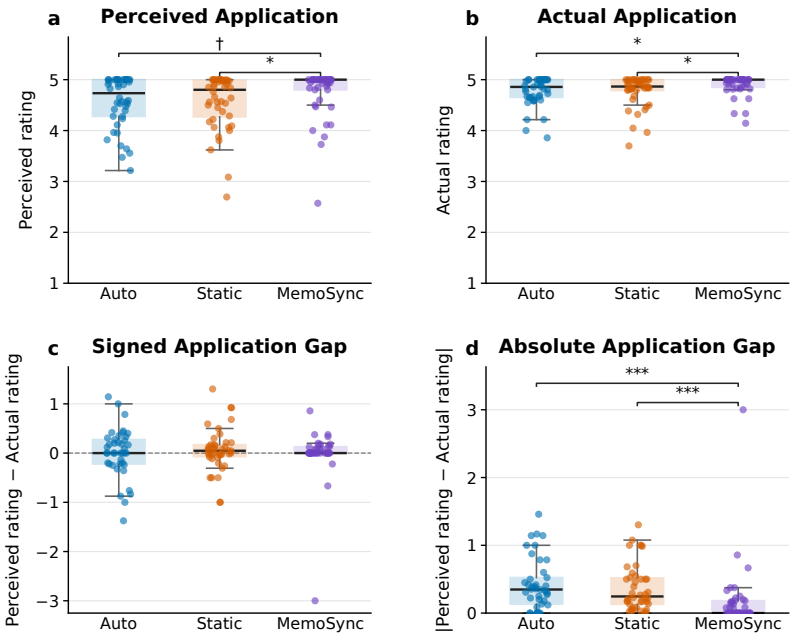


Fig. 12. Memory application perception. (a) Perceived application rating. (b) Actual application rating. (c) Signed gap (perceived minus actual). (d) Absolute gap. MemoSync users had the smallest gap between perceived and actual memory application. Brackets show all three pairwise Mann–Whitney U tests with Holm correction within each measure. $^{***}p < .001$, $^{*}p < .05$, $^{\dagger}p < .10$.

[9.18, 30.63]; vs. Auto: $ab = +11.0$ pp, BCa 95% CI [0.01, 21.50]).[2] The corresponding indirect effects through Control Accuracy were not significant (vs. Auto: $ab = -1.14$, CI [−3.29, 0.08]; vs. Static: $ab = -0.67$, CI [−2.38, 0.09]). After both mediators were included, neither residual direct effect reached $p < .05$ (vs. Static: $c' = +4.76$ pp, $p = .089$; vs. Auto: $c' = +1.85$ pp, $p = .513$). This exploratory pattern aligns with the monitoring-primacy principle in the metamemory literature: effective monitoring is a prerequisite for effective control, and the quality of control is bounded by the accuracy of monitoring [16, 23].

Finding 4: MemoSync reduced the gap between perceived and actual memory application. The absolute gap between perceived and evaluator-rated memory application under MemoSync ($M = 0.19$ on a 1–5 scale) was less than half of that under Auto ($M = 0.42$; $p < .001$, $r_{rb} = -.52$) and Static ($M = 0.38$; $p < .001$, $r_{rb} = -.56$; Figure 12d), while no significant difference was detected between Auto and Static. Signed gaps were near zero in every condition (Figure 12c), indicating no systematic over- or under-estimation. All three conditions allowed participants to view stored memory items, but only MemoSync displayed which items the agent selected and referenced during each generation turn and whether each item was followed, unused, or violated in the resulting interaction. We attribute the reduced memory application gap to this execution-level transparency, which enabled participants to directly observe how memory shaped agent outputs rather than relying on inference from the outputs alone.

[2]For the MemoSync–Auto contrast, the a path (condition → Monitoring Accuracy) reached $p = .052$, yet the bootstrapped ab CI excluded zero. Modern mediation inference treats the bootstrap CI for the indirect effect as the definitive criterion; neither individual-path significance nor total-effect significance is required [11, 21, 36].

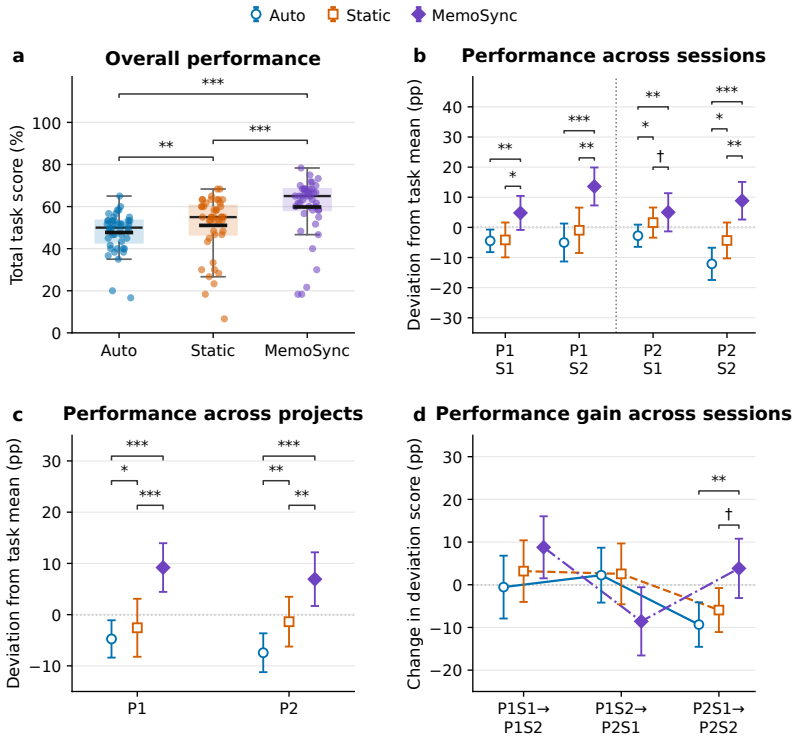


Fig. 13. Task performance. (a) Overall performance as a percentage of the 30-point maximum. (b) Per-session deviation from the corresponding task mean. (c) Mean deviation across the two sessions within each project. (d) Change in task-adjusted performance across each pair of consecutive sessions. In (a), boxes show medians and IQRs and black bars show means. In (b–d), markers show condition means and error bars show 95% CIs; lines in (d) connect the three ordered transitions within each condition. Brackets show all three pairwise Mann–Whitney U tests with Holm correction within each panel, session, project, or transition position. $n = 44$ per condition. $^{***}p < .001$, $^{**}p < .01$, $^{*}p < .05$, $^{\dagger}p < .10$.

4.2 Task Performance across Projects and Sessions

This subsection examines how memory co-management affected task performance. We first report overall and per-project condition differences, then trace how performance evolved across sessions, and finally discuss the relationship between memory alignment and task performance.

Finding 5: MemoSync achieved significantly higher task performance than both baselines; Static significantly outperformed Auto. MemoSync participants scored 59.8% of the maximum ($SD = 14.2$), significantly higher than Auto ($M = 47.7\%$, $SD = 9.3$; $p < .001$, $r_{rb} = .68$) and Static ($M = 51.1\%$, $SD = 14.3$; $p < .001$, $r_{rb} = .49$; Figure 13a). Static was also significantly higher than Auto ($p = .006$, $r_{rb} = -.34$). At the session level (Figure 13b), MemoSync remained above the task mean across all four sessions, significantly higher than both baselines at P1S1, P1S2, and P2S2, and significantly higher than Auto at P2S1 ($p = .004$, $r_{rb} = .40$) with a near-significant advantage over Static ($p = .073$, $r_{rb} = .22$). Averaged across the two sessions within each project (Figure 13c), MemoSync outperformed both baselines at P1 ($r_{rb} = .64$ and .45) and P2 ($r_{rb} = .61$ and .38), and Static outperformed Auto at both projects. Static's higher performance likely reflects the structural similarity between the two projects: the large volume of

development context accumulated in the Static configuration file could be reused across similar tasks, even when it contained scope-mismatched content.

Finding 6: At the final session (P2S2), MemoSync's task-adjusted performance continued to rise while both baselines declined. P2S1→P2S2 was the only transition where condition differences emerged (Figure 13d): MemoSync gained +3.8 pp, whereas Auto declined by 9.3 pp ($p = .010$, $r_{rb} = .36$); the difference from Static (−5.9 pp) approached significance ($p = .075$, $r_{rb} = .26$). The first two transitions did not differ by condition (all adjusted $p \geq .204$, all $|r_{rb}| \leq .23$). We conjecture that as sessions progressed, agent memory required ongoing maintenance to remain effective. Neither Auto nor Static created maintenance triggers within the interaction flow: Auto extracted memory automatically but did not clean outdated entries, and Static relied on users to edit the file without prompting which content needed updating. MemoSync embedded maintenance into routine interaction through system-initiated proposals, where the system proposed memory changes and users completed cleanup and updating through the approve-or-reject process, keeping memory current and relevant across sessions.

Finding 7: Memory alignment and task performance traded off between Auto and Static; MemoSync achieved the highest on both. Auto participants had higher memory alignment than Static (Finding 1), yet their task performance was significantly lower (Finding 5). Auto's automatic extraction mechanism imposed a storage cap, retaining high-level summaries; detailed but reusable information accumulated during sessions (e.g., environment-specific debugging approaches, concrete API call patterns) could be compressed during summarization or replaced once the cap was reached, leaving the agent without directly reusable references in later sessions. Static imposed no such cap, and developers and the agent continuously added context to the shared Markdown file, providing the agent with a richer pool of references that could be reused across structurally similar projects, even though developers gradually lost track of what the file contained (Figure 10). MemoSync resolved this trade-off by embedding monitoring and control across the full memory lifecycle: memory remained rich through continuous accumulation and cross-project transfer, while the hybrid-initiative proposal mechanism kept developers aware of what the agent retained and in what scope, achieving the highest alignment and the highest performance simultaneously.

4.3 Interaction Activity, Workload, and Usability

Finding 8: MemoSync participants performed significantly more monitoring and control actions, yet no condition differences were detected in workload, usability, or task time. MemoSync participants performed on average 70.3 monitoring actions over the four sessions ($SD = 43.9$), compared with 7.3 under Auto (IRR = 9.64, 95% CI [6.20, 14.97], $p_{\mathrm{H}} < .001$) and 8.7 under Static (IRR = 8.06, 95% CI [5.25, 12.36], $p_{\mathrm{H}} < .001$; Figure 14a). They also performed 25.3 control actions ($SD = 11.1$), compared with fewer than one under Auto (IRR = 30.14, 95% CI [13.32, 68.20], $p_{\mathrm{H}} < .001$) and Static (IRR = 101.36, 95% CI [40.32, 254.82], $p_{\mathrm{H}} < .001$; Figure 14b). We detected no condition differences in monitoring workload, control workload, system usability, or active task time (all omnibus $p \geq .200$, all $\epsilon^2 \leq .01$; Figure 14c–f). As described in the Introduction, developer-agent collaboration splits the meta-level and object-level of traditional metamemory apart, delegating object-level memory operations to the agent. Under Auto and Static, users had to self-initiate checks on this external object-level state. MemoSync's hybrid-initiative design surfaced object-level state changes at each interaction point, allowing the monitoring-control loop to operate without requiring users to initiate across this boundary. We conjecture that this design allowed the substantially increased activity to proceed without a perceived increase in workload.

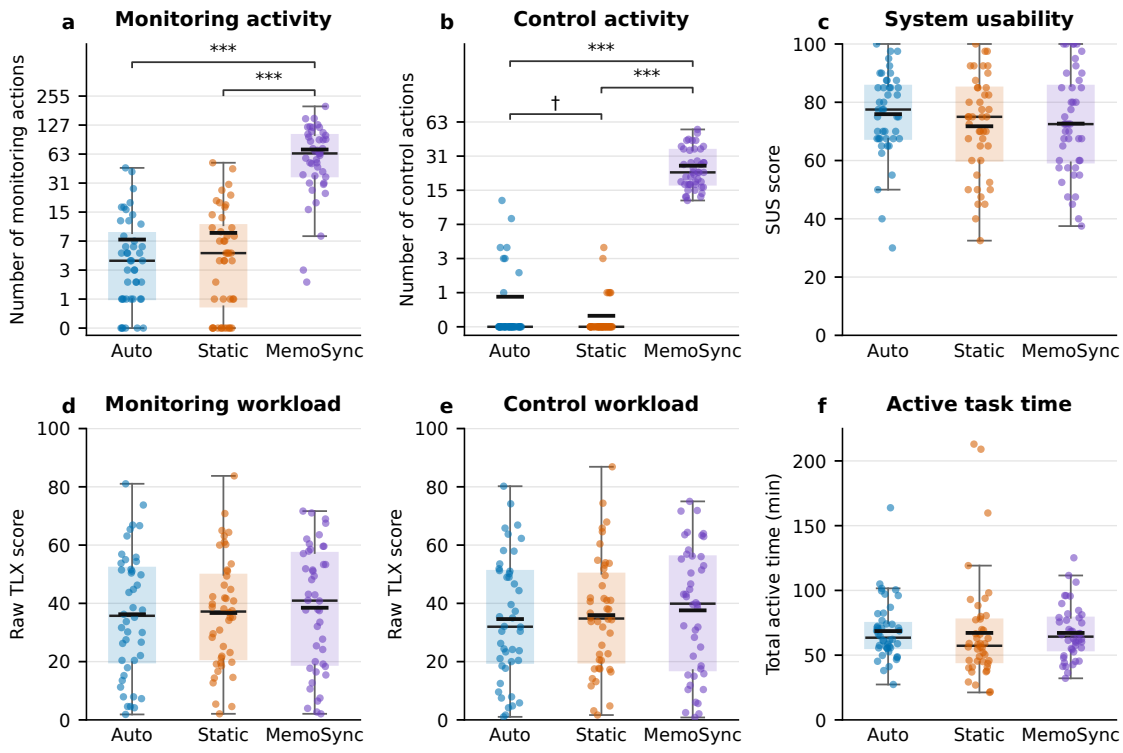


Fig. 14. Behavioral measures and cost. (a) Monitoring activity (log scale). (b) Control activity (log scale). (c) System usability (SUS). (d) Monitoring workload (Raw TLX). (e) Control workload (Raw TLX). (f) Active task time. MemoSync users performed significantly more monitoring and control actions; we detected no condition differences in monitoring workload, control workload, SUS, or active task time. Brackets show all three pairwise contrasts with Holm correction within each measure. $^{***}p < .001$, $^{\dagger}p < .10$.

References

[1] Anthropic. 2025. How Claude Remembers Your Project. https://code.claude.com/docs/en/memory.

[2] Anysphere. 2025. Cursor 1.0 Changelog: Bugbot, Background Agent, and Memories. https://cursor.com/changelog/1-0.

[3] Guangzong Cai, Ruiyin Li, Peng Liang, Zengyang Li, and Mojtaba Shahin. 2026. Rule Taxonomy and Evolution in AI IDEs: A Mining and Survey Study. *arXiv preprint arXiv:2606.12231* (2026).

[4] Worawalan Chatlatanagulchai, Kundjanasith Thonglek, Brittany Reid, Yutaro Kashiwa, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, and Hajimu Iida. 2025. On the use of agentic coding manifests: An empirical study of claude code. In *International Conference on Product-Focused Software Process Improvement*. Springer, 543–551.

[5] Cheng Chen, Maria D. Molina, Mengqi Liao, and Eugene Cho Snyder. 2026. Relational Gains, Privacy Strains: Exploring Users' Perceptions and Experiences with ChatGPT's Memory Feature. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*. ACM, New York, NY, USA. doi:10.1145/3772318.3791635

[6] Valerie Chen, Ameet Talwalkar, Robert Brennan, and Graham Neubig. 2026. Code with Me or for Me? How Increasing AI Automation Transforms Developer Workflows. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*. Association for Computing Machinery, New York, NY, USA, Article 122, 19 pages. doi:10.1145/3772318.3790850

[7] Xinyue Chen, Varun Manjunatha, Xu Wang, and Alexa Siu. 2026. From Conversation to Human-AI Common Ground: Extracting Cognitive Workflows for Reuse in Sense-making Tasks. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*. Association for Computing Machinery, New York, NY, USA, Article 1079, 23 pages. doi:10.1145/3772318.3791669

[8] Guangrui Fan, Dandan Liu, Lihu Pan, and Rui Zhang. 2026. When Help Hurts: Verification Load and Fatigue with AI Coding Assistants. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*. Association for Computing Machinery, New York, NY, USA, Article 733, 25 pages. doi:10.1145/3772318.3791176

[9] Franz Faul, Edgar Erdfelder, Axel Buchner, and Albert-Georg Lang. 2009. Statistical Power Analyses Using G*Power 3.1: Tests for Correlation and Regression Analyses. *Behavior Research Methods* 41, 4 (2009), 1149–1160.

[10] Andrew F Hayes. 2018. *Introduction to Mediation, Moderation, and Conditional Process Analysis: A Regression-Based Approach* (2nd ed.). Guilford Press, New York.

[11] Andrew F Hayes et al. 2013. Introduction to mediation, moderation, and conditional process analysis.

[12] Ruanqianqian Huang, Avery Reyna, Sorin Lerner, Haijun Xia, and Brian Hempel. 2025. Professional Software Developers Don't Vibe, They Control: AI Agent Use for Coding in 2025. arXiv preprint arXiv:2512.14012. arXiv:2512.14012 [cs.HC] doi:10.48550/arXiv.2512.14012

[13] Ziheng Huang, Sebastian Gutierrez, Hemanth Kamana, and Stephen MacNeil. 2023. Memory Sandbox: Transparent and Interactive Memory Management for Conversational Agents. In *Adjunct Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23 Adjunct)*. ACM, New York, NY, USA, 1–3. doi:10.1145/3586182.3615796

[14] Shaokang Jiang and Daye Nam. 2025. Beyond the Prompt: An Empirical Study of Cursor Rules. *arXiv preprint arXiv:2512.18925* (2025).

[15] Brennan Jones, Kelsey Stemmler, Emily Su, Young-Ho Kim, and Anastasia Kuzminykh. 2025. Users' Expectations and Practices with Agent Memory. In *Extended Abstracts of the 2025 CHI Conference on Human Factors in Computing Systems (CHI EA '25)*. ACM, New York, NY, USA, 8 pages. doi:10.1145/3706599.3720158

[16] Asher Koriat and Morris Goldsmith. 1996. Monitoring and control processes in the strategic regulation of memory accuracy. *Psychological review* 103, 3 (1996), 490.

[17] Ekaterina Koshchenko, Jovana Stankovic, Ilya Zakharov, and Agnia Sergeyuk. 2026. Configurable AI Coding Assistants: Designing For Developers Who Like to Be in Control. In *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering (FSE Companion '26)*. Association for Computing Machinery, New York, NY, USA, 1823–1831. doi:10.1145/3803437.3806709

[18] J Richard Landis and Gary G Koch. 1977. The Measurement of Observer Agreement for Categorical Data. *Biometrics* (1977), 159–174.

[19] Jenny T Liang, Chenyang Yang, and Brad A Myers. 2024. A large-scale survey on the usability of ai programming assistants: Successes and challenges. In *Proceedings of the 46th IEEE/ACM international conference on software engineering*. 1–13.

[20] Zimu Lu, Houxing Ren, Yunqiao Yang, Ke Wang, Zhuofan Zong, Mingjie Zhan, and Hongsheng Li. 2026. FullStack-Agent: Enhancing Agentic Full-Stack Web Coding via Development-Oriented Testing and Repository Back-Translation. *arXiv preprint arXiv:2602.03798* (2026).

[21] David Peter MacKinnon. 2008. *Introduction to statistical mediation analysis*. Routledge.

[22] Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz. 2024. Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems (CHI '24)*. ACM, New York, NY, USA, Article 142, 16 pages. doi:10.1145/3613904.3641936

[23] Thomas O Nelson. 1990. Metamemory: A theoretical framework and new findings. In *Psychology of learning and motivation*. Vol. 26. Elsevier, 125–173.

[24] OpenAI. 2025. Memories. Codex documentation. https://developers.openai.com/codex/memories

[25] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)*. ACM, New York, NY, USA, 22 pages. doi:10.1145/3586183.3606763

[26] Agnia Sergeyuk, Eric Huang, Dariia Karaeva, Anastasiia Serova, Yaroslav Golubev, and Iftekhar Ahmed. 2026. Evolving with AI: A Longitudinal Analysis of Developer Logs. *arXiv preprint arXiv:2601.10258* (2026).

[27] Lev Tankelevitch, Viktor Kewenig, Auste Simkute, Ava Elizabeth Scott, Advait Sarkar, Abigail Sellen, and Sean Rintel. 2024. The Metacognitive Demands and Opportunities of Generative AI. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems (CHI '24)*. ACM, New York, NY, USA, Article 680, 24 pages. doi:10.1145/3613904.3642902

[28] David R Thomas. 2006. A general inductive approach for analyzing qualitative evaluation data. *American journal of evaluation* 27, 2 (2006), 237–246.

[29] Christoph Treude and Sebastian Baltes. 2026. Context Rot in AI-Assisted Software Development: Repurposing Documentation Consistency for AI Configuration Artifacts. *arXiv preprint arXiv:2606.09090* (2026).

[30] Priyan Vaithilingam, Munyeong Kim, Frida-Cecilia Acosta-Parenteau, Daniel Lee, Amine Mhedhbi, Elena L. Glassman, and Ian Arawjo. 2025. Semantic Commit: Helping Users Update Intent Specifications for AI Memory at Scale. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology (UIST '25)*. ACM, New York, NY, USA, 18 pages. doi:10.1145/3746059.3747778

[31] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems (CHI EA '22)*. ACM, New York, NY, USA, Article 332, 7 pages. doi:10.1145/3491101.3519665

[32] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. Agent Workflow Memory. In *Forty-second International Conference on Machine Learning*. https://openreview.net/forum?id=NTAhi2JEEE

[33] Zidi Xiong, Yuping Lin, Wenya Xie, Pengfei He, Zirui Liu, Jiliang Tang, Himabindu Lakkaraju, and Zhen Xiang. 2026. How Memory Management Impacts LLM Agents: An Empirical Study of Experience-Following Behavior. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL '26)*. Association for Computational Linguistics, 623–645. doi:10.18653/v1/2026.acl-long.27

[34] Ryan Yen and Jian Zhao. 2024. Memolet: Reifying the Reuse of User-AI Conversational Memories. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology (UIST '24)*. ACM, New York, NY, USA, Article 58, 22 pages. doi:10.1145/3654777.3676388

Table 2. Participants in our formative study ($N = 12$): Coding gives years of programming experience, and Agent Use gives how long the participant has used AI coding agents. Projects is the number of projects in which the participant uses an agent on a sustained basis, and Sessions is the approximate number of agent sessions on their most-used project. Languages lists each participant's main programming languages, and Project Context gives the domain of their recent agent-assisted projects.

P#	Role	Age	Coding	Agent Use	Projects	Sessions	Languages	Project Context
P1	PhD Student	24	3–5	3–6 months	2	2–3	Python, Shell	Visualization
P2	Master's Student	23	3–5	>1 year	3	>10	Python, JS, C#, SQL	Interactive Tools
P3	Agent Engineer	28	>10	6–12 months	≥4	>10	JS, TS, Shell, Scala	Compilers, Agent
P4	PhD Student	23	6–10	>1 year	2	>10	Python, JS, C/C++	3D Generation
P5	PhD Student	25	6–10	>1 year	≥4	>10	Python	Super-Resolution
P6	Master's Student	23	3–5	6–12 months	2	4–10	Python, Shell	Graphics
P7	Software Engineer	30	6–10	>1 year	≥4	4–10	Python, TS, SQL	Full-Stack System
P8	LLM Engineer	23	3–5	<3 months	≥4	>10	Python	LLM Decoding
P9	Undergraduate	21	1–2	3–6 months	3	4–10	Python, Java	RAG
P10	Agent Engineer	24	3–5	6–12 months	2	>10	Python	Agent Workflow
P11	Software Engineer	27	>10	>1 year	3	>10	Python, JS, TS	LLM Inference
P12	ML Researcher	29	>10	6–12 months	≥4	>10	Python, SQL, Shell	Marketing Agent

[35] Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. 2025. A Survey on the Memory Mechanism of Large Language Model-based Agents. *ACM Transactions on Information Systems* 43, 6, Article 155 (Sept. 2025), 47 pages. doi:10.1145/3748302

[36] Xinshu Zhao, John G Lynch Jr, and Qimei Chen. 2010. Reconsidering Baron and Kenny: Myths and truths about mediation analysis. *Journal of consumer research* 37, 2 (2010), 197–206.

[37] Xiangrong Zhu, Jingwei Sun, Yuan Xu, Huanyi Wan, Jiawen Zhang, Yancheng Cao, Liuxin Zhang, Yu Zhang, Qianying Wang, and Xin Tong. 2026. MemoryLib: A Human-AI Collaborative Approach to Support Personalized Memory Meaning-Making Process. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 10, 2, Article 79 (June 2026), 47 pages. doi:10.1145/3810195

A Formative Study

A.1 Participant Demographics

B Formative Study Procedure Details

We provide more details of each portion of our formative study and its associated activities below.

B.1 Part A: Current Practices

We began by asking participants to describe their day-to-day collaboration with the AI coding agent: how a typical session starts and ends, what context they provide upfront, how much autonomy the agent has, and what prompts them to open a new session. We then asked about multi-project usage, including how many projects they work on with the agent, how often they switch between them, and whether they perceive the agent's state as isolated across projects. Finally, we asked about their use of configuration and memory files (e.g., `CLAUDE.md`, `.cursorrules`): how these files are created and maintained, how often they are reviewed, and whether participants understand how and when the agent reads them.

B.2 Part B: Artifact-Grounded Deep Dive

This was the core of the study. Participants opened their own materials (conversation logs, configuration files, and codebases from a recent project) and walked through them with the researcher. The walkthrough proceeded in five stages.

In *Memory Inventory*, participants identified information from their conversations and files that they considered worth persisting. They also flagged information they did not want the agent to retain. We asked them to describe each item in their own terms (e.g., project fact, personal preference, temporary workaround, lesson learned) without using researcher-defined categories. If participants had difficulty surfacing items, we used graduated scaffolding: asking first about task-level decisions, then about information they frequently re-explain, then about errors or risks, and finally walking through artifacts source by source.

In *Visibility Gap Detection*, we examined whether participants could determine, for each identified item, where it came from, whether the agent had access to it, how the agent might use it, and whether it could be edited or removed. We checked these four dimensions (awareness, comprehension, projection, and action) across the sources present in the participant's materials: configuration files, system-managed memory, conversation history, and code.

In *Organization and Scope Reasoning*, participants grouped their identified items and explained their rationale. We then asked them to assign each item to a scope level (personal, project, or session) and to reason about its abstraction: whether it should be stored as a specific value (e.g., "this project uses pnpm"), a conditional rule (e.g., "respect the project's existing package manager"), or a general principle. When participants struggled to group items, we used pairwise comparison and a scope ladder as scaffolding.

In *Lifecycle*, we asked participants to assess how long each persistent item would remain valid and what conditions would cause it to expire, such as a dependency upgrade, a branch merge, or a shift in team conventions. We also asked whether they had encountered the agent acting on outdated information, and how they currently detect and handle stale memory.

In *Cross-Project Transfer* (for participants working on multiple projects), we asked which items could move to a new project as-is, which would need rewriting to generalize, and which should not transfer at all. We asked participants to explain their judgment criteria and to describe any experiences with the agent incorrectly applying one project's memory in another.

B.3 Part C: Design Expectations

We concluded by asking participants about their preferences for future memory management support. Questions covered four areas: what the system should make visible and when; how candidate memories should be organized and presented; what management operations matter most (e.g., editing, scoping, merging, setting expiration); and how the system should help prevent and recover from errors caused by misapplied or stale memory. We asked participants to prioritize among candidate features and to reflect on the balance between proactive prevention and post-hoc correction.

C Formative Study Data Analysis

All sessions were audio-recorded and transcribed verbatim. Two researchers independently coded the full set of transcripts using an inductive approach [28]. In the first pass, each researcher open-coded the transcripts, generating initial codes grounded in participants' own language. Examples include "reuses debug lesson across projects," "surprised by auto-saved memory item," and "wants to review before merge." After independently coding the first four transcripts,

the two researchers met to compare codes, discuss disagreements, and merge overlapping codes into a shared codebook. This codebook was then applied to the remaining transcripts, with new codes added as needed and existing codes refined through weekly discussions.

After coding was complete, the researchers grouped related codes into higher-level categories. Four analytical dimensions emerged from this process: (1) what types of knowledge developers consider worth persisting and what criteria they use to decide (→ F1); (2) how developers structure memory along granularity, scope, and abstraction, and how these attributes affect cross-context portability (→ F2); (3) where visibility gaps prevent developers from verifying memory's influence on agent behavior (→ F3); and (4) how developers maintain memory over time and what level of control they prefer (→ F4). Participant counts (e.g., N=12) report the number of participants whose transcripts contained codes in a given category.

Table 3. Operationalization of the outcome measures. For Monitoring Accuracy, Control Accuracy, and Memory Alignment, we report Content, Scope, and Content+Scope, with Content+Scope as the primary metric. For Monitoring Accuracy and Memory Alignment, Scope is scored by comparing the corresponding personal/project/session scope labels. For Control Accuracy, s_i^R is a binary indicator of whether the observed application scope conforms to the desired scope. An item with an incorrect Scope receives zero on Content+Scope. Memory Application Gap is defined on Content only. $N = 132$ unless noted.

Measure	**Compares**	**Item-level scoring**	**Participant-level score**
Monitoring Accuracy	$A_i \leftrightarrow O_i$	Participants rated how completely they believed the agent remembered each item on a 1–5 scale, yielding Assessed Memory $A_i = (c_i^A, s_i^A)$. Object-Level Memory $O_i = (c_i^O, s_i^O)$ was obtained from the logged memory state. For Content, the item score is $1 - \lvert c_i^A - c_i^O \rvert / 4$. For Scope, s_i^A is compared with s_i^O; a match scores 1 and a mismatch scores 0. Content+Scope is the product of the Content and Scope scores.	Mean over scorable items ×100 (0–100%). Content, Scope, and Content+Scope are computed separately, with Content+Scope as the primary reported metric. Items marked as unknown by the participant are excluded.

Continued on next page

Table 3 continued from previous page

Measure	Compares	Item-level scoring	Participant-level score
Control Accuracy	$D_i \rightarrow R_i$	For each Desired Memory $D_i = (c_i^D, s_i^D)$, an LLM evaluator assessed its realization in the relevant frozen task outputs, yielding Actual Memory Application $R_i = (c_i^R, s_i^R)$. For Content, c_i^R is a 1–5 rating of how fully the desired content was realized, with item score $(c_i^R - 1)/4$. For Scope, the evaluator examined the item's application across the relevant session and project outputs and determined whether the observed application pattern conformed to the desired scope s_i^D; $s_i^R = 1$ if it did and $s_i^R = 0$ otherwise. Content+Scope is the product of the Content score and s_i^R. Not-applicable and uncertain items are excluded.	Mean over scorable items ×100 (0–100%). Content, Scope, and Content+Scope are computed separately, with Content+Scope as the primary reported metric; $N = 123$.
Memory Alignment	$D_i \leftrightarrow O_i$	Desired Memory $D_i = (c_i^D, s_i^D)$ is compared with Object-Level Memory $O_i = (c_i^O, s_i^O)$. For Content, the item score is $1 - \lvert c_i^D - c_i^O \rvert/4$, where c_i^D is the participant's 1–5 rating of how well the shown content matched their desired memory and c_i^O is the corresponding Object-Level Memory rating. For Scope, s_i^D is compared with s_i^O; a match scores 1 and a mismatch scores 0. Content+Scope is the product of the Content and Scope scores.	Mean over scorable items ×100 at each checkpoint (0–100%). Content, Scope, and Content+Scope are computed separately, with Content+Scope as the primary reported metric. Session-level Content+Scope Alignment is reported at P1S1, P1S2, P2S1, and P2S2; missing checkpoint scores remain missing.

Continued on next page

Table 3 continued from previous page

Measure	Compares	Item-level scoring	Participant-level score
Memory Application Gap	$P_i \leftrightarrow c_i^R$	Perceived Memory Application $P_i = (c_i^P)$ captures how fully participants believed each memory item was applied in the session output. The item score is the absolute difference between the participant's 1–5 application rating and the LLM evaluator's corresponding Actual Memory Application Content rating: $\lvert c_i^P - c_i^R \rvert$. This measure is defined on Content only.	Mean over matched items (0–4), with lower scores indicating more accurate perception of memory application; $N = 123$.
Task Performance	—	Each session was scored using the benchmark's official rubric, covering frontend, backend, and database requirements.	Sum across the four sessions, expressed as a percentage of the 30-point maximum. Deviation scores subtract the corresponding task-level mean. Task-adjusted performance gain is the difference between the later and earlier deviation scores for each consecutive transition: P1S1→P1S2, P1S2→P2S1, and P2S1→P2S2.
Monitoring / Control Activity	—	Monitoring and control actions were identified from the interaction logs.	Four-session totals for monitoring and control actions, respectively.
Monitoring / Control Workload	—	Raw NASA-TLX was administered after each session, anchored separately to monitoring and control.	Mean across the four sessions for monitoring and control, respectively (0–100).
System Usability	—	The ten-item SUS was administered once after all sessions.	Standard SUS score (0–100).

D Full Test Statistics

Table 4 reports the full non-count outcome statistics behind the current Results figures, Table 5 reports the negative-binomial models for the activity counts, and Table 6 reports every component of the parallel mediation model. In every table, each p value is reported alongside its corresponding effect estimate.

Table 4. Full test statistics for the outcomes shown in Figures 9, 10, 13, 14, and 12. Descriptives are M (SD). Each omnibus cell reports Kruskal–Wallis H, its p value, and ϵ^2. Each pairwise cell reports Mann–Whitney U, the Holm-adjusted p_{H}, and signed rank-biserial r_{rb}. Effect signs follow the contrast named in the column. Holm correction covers all three pairwise contrasts within each measure or position. $N = 132$ (44 per condition) unless noted.

Measure	Auto	Static	MemoSync	Omnibus $H; p; \epsilon^2$	Auto–Static $U; p_{\mathrm{H}}; r_{rb}$	Memo–Auto $U; p_{\mathrm{H}}; r_{rb}$	Memo–Static $U; p_{\mathrm{H}}; r_{rb}$
Figure 3: Monitoring, control, and alignment views							
Monitoring Accuracy: Content (%)	87.1 (9.0)	76.2 (13.9)	93.6 (13.0)	$H = 47.78$ $p =< .001$ $\epsilon^2 = .35$	$U = 1406$ $p_{\mathrm{H}} =< .001$ $r_{rb} = .45$	$U = 1548$ $p_{\mathrm{H}} =< .001$ $r_{rb} = .60$	$U = 1688.5$ $p_{\mathrm{H}} =< .001$ $r_{rb} = .74$
Monitoring Accuracy: Scope (%)	70.5 (29.8)	59.5 (27.8)	76.2 (28.3)	$H = 11.65$ $p = .003$ $\epsilon^2 = .07$	$U = 1215$ $p_{\mathrm{H}} = .077$ $r_{rb} = .26$	$U = 1155$ $p_{\mathrm{H}} = .116$ $r_{rb} = .19$	$U = 1359.5$ $p_{\mathrm{H}} = .003$ $r_{rb} = .40$
Monitoring Accuracy: Content+Scope (%)	66.2 (30.3)	56.3 (28.9)	74.3 (29.2)	$H = 11.1$ $p = .004$ $\epsilon^2 = .07$	$U = 1162$ $p_{\mathrm{H}} = .110$ $r_{rb} = .20$	$U = 1198$ $p_{\mathrm{H}} = .110$ $r_{rb} = .24$	$U = 1352.5$ $p_{\mathrm{H}} = .004$ $r_{rb} = .40$
Control Accuracy: Content (%)[a]	93.2 (7.4)	93.7 (7.7)	97.2 (5.1)	$H = 13.84$ $p =< .001$ $\epsilon^2 = .10$	$U = 825.5$ $p_{\mathrm{H}} = .749$ $r_{rb} = -.04$	$U = 1145$ $p_{\mathrm{H}} = .003$ $r_{rb} = .40$	$U = 1190.5$ $p_{\mathrm{H}} = .003$ $r_{rb} = .42$
Control Accuracy: Scope (%)[a]	93.0 (8.0)	93.4 (8.0)	96.9 (5.3)	$H = 11.96$ $p = .003$ $\epsilon^2 = .08$	$U = 859$ $p_{\mathrm{H}} = .989$ $r_{rb} = .00$	$U = 1112.5$ $p_{\mathrm{H}} = .009$ $r_{rb} = .36$	$U = 1177.5$ $p_{\mathrm{H}} = .004$ $r_{rb} = .40$
Control Accuracy: Content+Scope (%)[a]	89.9 (10.8)	92.2 (8.8)	95.5 (7.1)	$H = 9.61$ $p = .008$ $\epsilon^2 = .06$	$U = 795$ $p_{\mathrm{H}} = .549$ $r_{rb} = -.08$	$U = 1111$ $p_{\mathrm{H}} = .014$ $r_{rb} = .35$	$U = 1106$ $p_{\mathrm{H}} = .024$ $r_{rb} = .32$
Memory Alignment: Content (%)	77.6 (18.8)	66.2 (16.9)	88.0 (13.9)	$H = 35.86$ $p =< .001$ $\epsilon^2 = .26$	$U = 1367.5$ $p_{\mathrm{H}} = .002$ $r_{rb} = .41$	$U = 1348$ $p_{\mathrm{H}} = .002$ $r_{rb} = .39$	$U = 1651.5$ $p_{\mathrm{H}} =< .001$ $r_{rb} = .71$
Memory Alignment: Scope (%)	72.6 (27.6)	57.8 (26.9)	75.7 (29.3)	$H = 14.32$ $p =< .001$ $\epsilon^2 = .10$	$U = 1303$ $p_{\mathrm{H}} = .010$ $r_{rb} = .35$	$U = 1150$ $p_{\mathrm{H}} = .126$ $r_{rb} = .19$	$U = 1373$ $p_{\mathrm{H}} = .002$ $r_{rb} = .42$
Memory Alignment: Content+Scope (%)	61.8 (27.8)	50.0 (27.1)	69.0 (30.7)	$H = 11.58$ $p = .003$ $\epsilon^2 = .07$	$U = 1227$ $p_{\mathrm{H}} = .062$ $r_{rb} = .27$	$U = 1165$ $p_{\mathrm{H}} = .101$ $r_{rb} = .20$	$U = 1348$ $p_{\mathrm{H}} = .005$ $r_{rb} = .39$
Figure 4: Memory Alignment across sessions							
Alignment trajectory: Content P1S1 (%)[b]	73.3 (23.5)	76.6 (22.7)	90.7 (13.8)	$H = 11.96$ $p = .003$ $\epsilon^2 = .18$	$U = 137.5$ $p_{\mathrm{H}} = .689$ $r_{rb} = -.08$	$U = 350.5$ $p_{\mathrm{H}} = .003$ $r_{rb} = .59$	$U = 241$ $p_{\mathrm{H}} = .031$ $r_{rb} = .46$
Alignment trajectory: Content P1S2 (%)[b]	79.6 (17.4)	77.1 (20.0)	88.1 (17.5)	$H = 14.03$ $p =< .001$ $\epsilon^2 = .09$	$U = 997.5$ $p_{\mathrm{H}} = .808$ $r_{rb} = .03$	$U = 1394$ $p_{\mathrm{H}} = .001$ $r_{rb} = .44$	$U = 1312$ $p_{\mathrm{H}} = .007$ $r_{rb} = .36$
Alignment trajectory: Content P2S1 (%)[b]	77.2 (22.1)	44.5 (30.4)	85.7 (19.8)	$H = 39.17$ $p =< .001$ $\epsilon^2 = .32$	$U = 1514.5$ $p_{\mathrm{H}} =< .001$ $r_{rb} = .56$	$U = 986$ $p_{\mathrm{H}} = .003$ $r_{rb} = .40$	$U = 1219.5$ $p_{\mathrm{H}} =< .001$ $r_{rb} = .73$
Alignment trajectory: Content P2S2 (%)[b]	77.6 (20.2)	77.9 (18.4)	86.6 (15.2)	$H = 9.7$ $p = .008$ $\epsilon^2 = .06$	$U = 951$ $p_{\mathrm{H}} = .890$ $r_{rb} = -.02$	$U = 1280$ $p_{\mathrm{H}} = .013$ $r_{rb} = .35$	$U = 1243.5$ $p_{\mathrm{H}} = .021$ $r_{rb} = .31$

Continued on next page

Table 4 continued from previous page

Measure	Auto	Static	MemoSync	Omnibus H; p; ϵ^2	Auto–Static U; p_H; r_{rb}	Memo–Auto U; p_H; r_{rb}	Memo–Static U; p_H; r_{rb}
Alignment trajectory: Scope P1S1 (%)[b]	60.2 (34.7)	58.6 (33.3)	68.3 (44.3)	$H = 3.85$ $p = .146$ $\epsilon^2 = .03$	$U = 152.5$ $p_H = .946$ $r_{rb} = .02$	$U = 298$ $p_H = .138$ $r_{rb} = .35$	$U = 204.5$ $p_H = .413$ $r_{rb} = .24$
Alignment trajectory: Scope P1S2 (%)[b]	76.9 (25.8)	69.0 (32.6)	79.1 (33.5)	$H = 10.9$ $p = .004$ $\epsilon^2 = .07$	$U = 1020$ $p_H = .663$ $r_{rb} = .05$	$U = 1361$ $p_H = .002$ $r_{rb} = .41$	$U = 1246.5$ $p_H = .030$ $r_{rb} = .29$
Alignment trajectory: Scope P2S1 (%)[b]	69.7 (34.7)	41.8 (31.4)	68.0 (41.0)	$H = 15.99$ $p =< .001$ $\epsilon^2 = .12$	$U = 1391$ $p_H = .001$ $r_{rb} = .44$	$U = 870$ $p_H = .075$ $r_{rb} = .24$	$U = 986.5$ $p_H = .006$ $r_{rb} = .40$
Alignment trajectory: Scope P2S2 (%)[b]	73.2 (30.5)	65.1 (33.1)	77.1 (31.4)	$H = 8.74$ $p = .013$ $\epsilon^2 = .05$	$U = 1071.5$ $p_H = .384$ $r_{rb} = .11$	$U = 1219.5$ $p_H = .036$ $r_{rb} = .29$	$U = 1254$ $p_H = .023$ $r_{rb} = .33$
Alignment trajectory: Content+Scope P1S1 (%)[b]	49.0 (35.6)	50.4 (34.5)	64.1 (42.8)	$H = 3.03$ $p = .220$ $\epsilon^2 = .02$	$U = 143$ $p_H = .828$ $r_{rb} = -.05$	$U = 286$ $p_H = .285$ $r_{rb} = .30$	$U = 203$ $p_H = .473$ $r_{rb} = .23$
Alignment trajectory: Content+Scope P1S2 (%)[b]	65.6 (26.8)	60.3 (33.3)	71.2 (34.3)	$H = 4.85$ $p = .089$ $\epsilon^2 = .02$	$U = 1008.5$ $p_H = .738$ $r_{rb} = .04$	$U = 1205$ $p_H = .142$ $r_{rb} = .24$	$U = 1184$ $p_H = .142$ $r_{rb} = .22$
Alignment trajectory: Content+Scope P2S1 (%)[b]	59.9 (33.8)	33.8 (29.2)	60.0 (40.0)	$H = 12.62$ $p = .002$ $\epsilon^2 = .09$	$U = 1366.5$ $p_H = .003$ $r_{rb} = .41$	$U = 772$ $p_H = .476$ $r_{rb} = .10$	$U = 954.5$ $p_H = .017$ $r_{rb} = .36$
Alignment trajectory: Content+Scope P2S2 (%)[b]	62.3 (29.8)	57.8 (33.6)	71.0 (32.2)	$H = 5.71$ $p = .057$ $\epsilon^2 = .03$	$U = 1006.5$ $p_H = .751$ $r_{rb} = .04$	$U = 1192.5$ $p_H = .108$ $r_{rb} = .26$	$U = 1184.5$ $p_H = .108$ $r_{rb} = .25$

Figure 5: Task performance

Overall Performance (%)	47.7 (9.3)	51.1 (14.3)	59.8 (14.2)	$H = 35.57$ $p =< .001$ $\epsilon^2 = .26$	$U = 637$ $p_H = .006$ $r_{rb} = -.34$	$U = 1626.5$ $p_H =< .001$ $r_{rb} = .68$	$U = 1442.5$ $p_H =< .001$ $r_{rb} = .49$
Performance level: P1S1 (pp)	−4.5 (12.3)	−4.2 (19.0)	4.8 (18.6)	$H = 12.85$ $p = .002$ $\epsilon^2 = .08$	$U = 873.5$ $p_H = .430$ $r_{rb} = -.10$	$U = 1363.5$ $p_H = .003$ $r_{rb} = .41$	$U = 1298.5$ $p_H = .012$ $r_{rb} = .34$
Performance level: P1S2 (pp)	−5.0 (20.7)	−1.0 (24.8)	13.6 (20.7)	$H = 17.37$ $p =< .001$ $\epsilon^2 = .12$	$U = 825.5$ $p_H = .232$ $r_{rb} = -.15$	$U = 1442$ $p_H =< .001$ $r_{rb} = .49$	$U = 1317.5$ $p_H = .007$ $r_{rb} = .36$
Performance level: P2S1 (pp)	−2.8 (12.1)	1.6 (16.5)	5.0 (20.9)	$H = 12.82$ $p = .002$ $\epsilon^2 = .08$	$U = 680$ $p_H = .032$ $r_{rb} = -.30$	$U = 1354.5$ $p_H = .004$ $r_{rb} = .40$	$U = 1183$ $p_H = .073$ $r_{rb} = .22$
Performance level: P2S2 (pp)	−12.1 (17.6)	−4.3 (19.6)	8.8 (20.5)	$H = 23.74$ $p =< .001$ $\epsilon^2 = .17$	$U = 706.5$ $p_H = .027$ $r_{rb} = -.27$	$U = 1526$ $p_H =< .001$ $r_{rb} = .58$	$U = 1317$ $p_H = .007$ $r_{rb} = .36$
Project mean: P1 (pp)	−4.7 (12.0)	−2.6 (18.6)	9.2 (15.6)	$H = 29.36$ $p =< .001$ $\epsilon^2 = .21$	$U = 726.5$ $p_H = .044$ $r_{rb} = -.25$	$U = 1584$ $p_H =< .001$ $r_{rb} = .64$	$U = 1407.5$ $p_H =< .001$ $r_{rb} = .45$

Continued on next page

Table 4 continued from previous page

Measure	Auto	Static	MemoSync	Omnibus H; p; ϵ^2	Auto–Static U; p_H; r_{rb}	Memo–Auto U; p_H; r_{rb}	Memo–Static U; p_H; r_{rb}
Project mean: P2 (pp)	−7.4 (12.5)	−1.4 (16.0)	6.9 (17.2)	$H = 27.58$ $p =< .001$ $\epsilon^2 = .20$	$U = 627$ $p_H = .005$ $r_{rb} = -.35$	$U = 1556.5$ $p_H =< .001$ $r_{rb} = .61$	$U = 1333.5$ $p_H = .005$ $r_{rb} = .38$
Gain: P1S1→P1S2 (pp)	−0.5 (24.2)	3.2 (23.7)	8.8 (23.9)	$H = 3.32$ $p = .191$ $\epsilon^2 = .01$	$U = 885$ $p_H = .491$ $r_{rb} = -.09$	$U = 1179$ $p_H = .237$ $r_{rb} = .22$	$U = 1108.5$ $p_H = .485$ $r_{rb} = .15$
Gain: P1S2→P2S1 (pp)	2.2 (21.1)	2.6 (23.4)	−8.6 (26.3)	$H = 4.03$ $p = .133$ $\epsilon^2 = .02$	$U = 985.5$ $p_H = .887$ $r_{rb} = .02$	$U = 749$ $p_H = .204$ $r_{rb} = -.23$	$U = 772.5$ $p_H = .207$ $r_{rb} = -.20$
Gain: P2S1→P2S2 (pp)	−9.3 (17.1)	−5.9 (17.0)	3.8 (22.9)	$H = 9.5$ $p = .009$ $\epsilon^2 = .06$	$U = 837$ $p_H = .276$ $r_{rb} = -.14$	$U = 1321$ $p_H = .010$ $r_{rb} = .36$	$U = 1217.5$ $p_H = .075$ $r_{rb} = .26$
Figure 7: Workload, usability, and time							
Monitoring Workload (Raw TLX)	36.2 (21.4)	36.7 (19.2)	38.5 (21.1)	$H = 0.38$ $p = .829$ $\epsilon^2 = .00$	$U = 963.5$ $p_H = 1.000$ $r_{rb} = .00$	$U = 1035.5$ $p_H = 1.000$ $r_{rb} = .07$	$U = 1027$ $p_H = 1.000$ $r_{rb} = .06$
Control Workload (Raw TLX)	34.6 (21.4)	36.0 (19.9)	37.6 (21.9)	$H = 0.52$ $p = .771$ $\epsilon^2 = .00$	$U = 934$ $p_H = 1.000$ $r_{rb} = -.04$	$U = 1050.5$ $p_H = 1.000$ $r_{rb} = .09$	$U = 1024.5$ $p_H = 1.000$ $r_{rb} = .06$
SUS	75.9 (14.7)	71.8 (17.0)	72.7 (17.7)	$H = 1.34$ $p = .511$ $\epsilon^2 = .00$	$U = 1093$ $p_H = .893$ $r_{rb} = .13$	$U = 854.5$ $p_H = .893$ $r_{rb} = -.12$	$U = 982$ $p_H = .910$ $r_{rb} = .01$
Active Task Time (min)	68.6 (23.3)	67.2 (41.0)	67.2 (20.7)	$H = 3.22$ $p = .200$ $\epsilon^2 = .01$	$U = 1165$ $p_H = .303$ $r_{rb} = .20$	$U = 943$ $p_H = .838$ $r_{rb} = -.03$	$U = 1140$ $p_H = .305$ $r_{rb} = .18$
Figure 8: Application perception							
Perceived Application (1–5)	4.56 (0.51)	4.56 (0.54)	4.76 (0.48)	$H = 8.54$ $p = .014$ $\epsilon^2 = .05$	$U = 1007$ $p_H = .745$ $r_{rb} = .04$	$U = 1225$ $p_H = .050$ $r_{rb} = .27$	$U = 1295.5$ $p_H = .015$ $r_{rb} = .34$
Actual Application (1–5)[a]	4.77 (0.28)	4.77 (0.31)	4.89 (0.20)	$H = 8.55$ $p = .014$ $\epsilon^2 = .05$	$U = 846.5$ $p_H = .897$ $r_{rb} = -.02$	$U = 1073$ $p_H = .027$ $r_{rb} = .31$	$U = 1112$ $p_H = .027$ $r_{rb} = .32$
Signed Application Gap[a]	0.01 (0.49)	0.06 (0.44)	−0.01 (0.53)	$H = 0.03$ $p = .985$ $\epsilon^2 = .00$	$U = 867.5$ $p_H = 1.000$ $r_{rb} = .01$	$U = 815$ $p_H = 1.000$ $r_{rb} = -.01$	$U = 865$ $p_H = 1.000$ $r_{rb} = .03$
Absolute Application Gap[a]	0.42 (0.38)	0.38 (0.34)	0.19 (0.49)	$H = 23.65$ $p =< .001$ $\epsilon^2 = .18$	$U = 904.5$ $p_H = .695$ $r_{rb} = .05$	$U = 395.5$ $p_H =< .001$ $r_{rb} = -.52$	$U = 371$ $p_H =< .001$ $r_{rb} = -.56$

[a] $N = 123$ (41 Auto, 42 Static, 40 MemoSync); participants without scorable items are excluded.

[b] Observed n for Auto/Static/MemoSync: P1S1 20/15/22; P1S2 44/44/44; P2S1 44/44/32; P2S2 44/44/43.

Table 5. Negative-binomial models for the activity counts. Descriptives are M (SD). IRR = incidence rate ratio for the first condition over the second, with HC3 robust 95% CIs. All three pairwise contrasts are Holm-corrected within each activity measure.

	Descriptives			Omnibus	Auto/Static		Memo/Auto		Memo/Static	
Measure	Auto	Static	MemoSync	LR $\chi^2(2)$; p	IRR [95% CI]	p_H	IRR [95% CI]	p_H	IRR [95% CI]	p_H
Monitoring Activity	7.3 (10.0)	8.7 (11.6)	70.3 (43.9)	87.38; <.001	0.84 [0.48, 1.46]	.529	9.64 [6.20, 14.97]	<.001	8.06 [5.25, 12.36]	<.001
Control Activity	0.8 (2.3)	0.3 (0.8)	25.3 (11.1)	139.52; <.001	3.36 [0.99, 11.37]	.051	30.14 [13.32, 68.20]	<.001	101.36 [40.32, 254.82]	<.001

Table 6. Complete component statistics for the parallel mediation model in Figure 11 (Memory Alignment, Content+Scope; $N = 123$). Path, residual-direct, and total-effect rows report unstandardized OLS coefficients B, HC3 95% CIs, HC3 z statistics, and two-sided p values. Indirect rows report the unstandardized product ab and BCa 95% CIs from 50,000 condition-stratified participant bootstrap resamples; no separate point-null p value is used for these rows, and exclusion of zero by the BCa CI is the inferential criterion.

Effect	Contrast	Estimate	95% CI	z	p
a_1: condition → Monitoring Accuracy	MemoSync–Auto	$B = +12.57$	$[-.09, 25.23]$	1.95	.052
a_1: condition → Monitoring Accuracy	MemoSync–Static	$B = +22.96$	$[10.82, 35.11]$	3.71	$< .001$
a_2: condition → Control Accuracy	MemoSync–Auto	$B = +5.60$	$[1.59, 9.61]$	2.74	.006
a_2: condition → Control Accuracy	MemoSync–Static	$B = +3.28$	$[-.21, 6.77]$	1.84	.065
b_1: Monitoring Accuracy → alignment	Common to both contrasts	$B = +.88$	$[.77, .99]$	15.62	$< .001$
b_2: Control Accuracy → alignment	Common to both contrasts	$B = -.20$	$[-.50, .10]$	−1.33	.184
Indirect via Monitoring Accuracy	MemoSync–Auto	$ab = +11.03$	$[.01, 21.50]$	–	–
Indirect via Monitoring Accuracy	MemoSync–Static	$ab = +20.15$	$[9.18, 30.63]$	–	–
Indirect via Control Accuracy	MemoSync–Auto	$ab = -1.14$	$[-3.29, .08]$	–	–
Indirect via Control Accuracy	MemoSync–Static	$ab = -.67$	$[-2.38, .09]$	–	–
Total indirect effect	MemoSync–Auto	$ab = +9.89$	$[-.92, 20.24]$	–	–
Total indirect effect	MemoSync–Static	$ab = +19.48$	$[8.57, 30.02]$	–	–
Residual direct effect (c')	MemoSync–Auto	$B = +1.85$	$[-3.70, 7.40]$	.65	.513
Residual direct effect (c')	MemoSync–Static	$B = +4.76$	$[-.73, 10.26]$	1.70	.089
Total effect (c)	MemoSync–Auto	$B = +11.74$	$[-.56, 24.05]$	1.87	.061
Total effect (c)	MemoSync–Static	$B = +24.25$	$[12.23, 36.26]$	3.95	$< .001$

E Technical Evaluation

We adapted the apartment-rental task (038) and the car-rental task (098) from FullStack-Bench and split each into two consecutive development sessions along the official task description. In the apartment-rental project, Session 1 implements listing browsing, search, and filtering, and Session 2 adds booking and booking records; in the car-rental project, Session 1 implements vehicle browsing and booking, and Session 2 adds payment and order management. Session 2 continues in the same workspace on the code accumulated in Session 1.

These runs serve two purposes: they calibrate task difficulty, and they evaluate the capability of the coding agent itself. We ran the four sessions in a fixed order with no accumulated memory, repeated independently ten times (40 sessions in total). The agent was Claude Code (v2.1.170) with DeepSeek V4 Flash at the extra-high reasoning setting. After each session, the workspace was frozen and scored by the original FullStack-Bench evaluator, using GPT-5.6 Sol as the text and vision judge. Session 1 was scored on its own official items only. Session 2 received the full official evaluation over the cumulative workspace, from which we extracted the Session 2 items; this evaluation also covers regressions in Session 1 functionality and the final state of the whole project. Scores sum the Frontend, Backend, and DB Structure items. Table 7 reports the mean score per session; the mean scoring rate across the four sessions was approximately 50%, leaving room to observe both gains and losses from memory management.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009

Table 7. Agent-only calibration scores (ten runs per session, no accumulated memory).

Session	Mean score / max	Scoring rate
Apartment-rental, Session 1	4.30 / 9	47.8%
Apartment-rental, Session 2	2.40 / 5	48.0%
Car-rental, Session 1	5.75 / 11	52.3%
Car-rental, Session 2	2.50 / 5	50.0%